

claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf\_49c1018a-e67\agent-ae692a1ec6051ce22.jsonl`
- SHA-256: `ac6a2d5f42199c22aa8613e22e6ea8c41423c8fbbdb2183e8a19274ec248b4044`
- Source modified: `2026-06-18T01:36:16+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf\_49c1018a-e67`
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`

Transcript

1. user (2026-06-18T01:33:12.224Z)

Adversarial bug-hunter, mature Python repo (badminton-highlight-indexer). Read READ-ONLY in this worktree (origin/master), DO NOT modify:

```
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/hunt-base
```

A prior precision pass found ZERO bugs; you cross-cut by BUG CLASS across ALL files to catch what an area-by-area pass missed. Report a finding ONLY if you READ the code and a CONCRETE trigger reaches it and a maintainer would call it a real defect. Include exact file+line, the trigger, why it's wrong, severity, minimal fix, confidence. PRECISION over recall ? empty list is a valid, respected answer; do NOT manufacture findings. EXCLUDE backend/eval/ablation.py, backend/eval/fusion_compare.py, backend/eval/fusion_audio.py, scripts/nightly_regression.py.

CLASS: CLI/argparse + FastAPI request handling.

Wrong/again-mutated argparse defaults, exit-code correctness, unvalidated request fields, type coercion, missing 404/400 paths, mutable default args, off-by-one in pagination/range params.

2. assistant (2026-06-18T01:33:14.660Z)

I'll hunt for bugs in the CLI/argparse + FastAPI request handling class. Let me start by finding the relevant files.

3. user (2026-06-18T01:33:15.821Z)

4bd9d2d fix(eval): ablation litmus tolerates a mixed-schema golden corpus (#209)
Not currently on any branch.
nothing to commit, working tree clean

4. assistant (2026-06-18T01:33:18.137Z)

Let me find the CLI/argparse and FastAPI files.

5. user (2026-06-18T01:33:20.918Z)

Found 28 files

```
.claude\worktrees\hunt-base\training\gen0\harness.py
.claude\worktrees\hunt-base\tools\generate_highlights.py
.claude\worktrees\hunt-base\tools\cleanup_caches.py
.claude\worktrees\hunt-base\tests\test_golden_real_fixtures.py
.claude\worktrees\hunt-base\scripts\run_phase3_eval.py
.claude\worktrees\hunt-base\scripts\nightly_regression.py
.claude\worktrees\hunt-base\run_eval.py
.claude\worktrees\hunt-base\run_regression.py
.claude\worktrees\hunt-base\docs\CODE_MAP.md
.claude\worktrees\hunt-base\backend\tools\rally_slicer.py
.claude\worktrees\hunt-base\backend\pipeline\detectors\wasb_infer.py
```

```
.claude\worktrees\hunt-base\backend\main.py
.claude\worktrees\hunt-base\backend\eval\serve_contrast.py
.claude\worktrees\hunt-base\backend\eval\served_gate_a.py
.claude\worktrees\hunt-base\backend\eval\rally_seq_proto.py
.claude\worktrees\hunt-base\backend\eval\rally_seg_eval.py
.claude\worktrees\hunt-base\backend\eval\golden_fixtures.py
.claude\worktrees\hunt-base\backend\eval\fusion_golden.py
.claude\worktrees\hunt-base\backend\eval\gemini_refine.py
.claude\worktrees\hunt-base\backend\eval\fusion_audio.py
.claude\worktrees\hunt-base\backend\eval\fusion_compare.py
.claude\worktrees\hunt-base\backend\eval\export_wasb_segments.py
.claude\worktrees\hunt-base\backend\eval\eval_partial_labels.py
.claude\worktrees\hunt-base\backend\eval\distill_local.py
.claude\worktrees\hunt-base\backend\eval\calibrate_wasb.py
.claude\worktrees\hunt-base\backend\eval\calibrate_local.py
.claude\worktrees\hunt-base\backend\eval\audio_e1_probe.py
.claude\worktrees\hunt-base\backend\eval\ablation.py
```

6. user (2026-06-18T01:33:21.152Z)

Found 5 files

```
.claude\worktrees\hunt-base\tests\test_upload_download.py
.claude\worktrees\hunt-base\tests\test_job_worker_loop.py
.claude\worktrees\hunt-base\tests\test_api_endpoints.py
.claude\worktrees\hunt-base\tests\test_async_jobs.py
.claude\worktrees\hunt-base\backend\main.py
```

7. assistant (2026-06-18T01:33:23.583Z)

The FastAPI app is in `backend/main.py`. Let me read it first as it's the core of the request handling.

8. user (2026-06-18T01:33:24.604Z)

```
1     import os
2     import sys
3     import argparse
4     import logging
5     import threading
6     import traceback
7     import uuid
8     from concurrent.futures import ThreadPoolExecutor
9     import uvicorn
10    from dotenv import load_dotenv
11
12    load_dotenv()
13
14    # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
15    logging.basicConfig(
16        level=logging.INFO,
17        format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
18        datefmt='%H:%M:%S'
19    )
20    logger = logging.getLogger(__name__)
21    from fastapi import FastAPI, HTTPException, Request
22    from fastapi.middleware.cors import CORSMiddleware
23    from pydantic import BaseModel
24    from typing import List, Dict, Any, Optional
25
26    from fastapi.staticfiles import StaticFiles
27    from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
28    from backend.pipeline.validators.base import registry
29    from backend.infrastructure.database import Database
```

```

30     from backend.pipeline.segmenters.base import segmenter_registry
31     from backend.pipeline.stitcher.compiler import VideoCompiler
32     from backend.utils.hashing import compute_video_id # canonical file-identity hashing
33     from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path
34     from backend.config import load_config as load_app_config, AppConfig, StitchingConfig #
new typed config
35     from backend.results import Failure # standardized error/result contract
36     from backend.reporting import emit_indexer_report
37
38     app = FastAPI(title="Sports Highlight Indexer API")
39
40     # Enable CORS for the local web interface. allow_origins='' with allow_credentials=True
is
41     # rejected by browsers per the fetch spec and is an unsafe default to carry into the
42     # auth/tenancy work; this tool uses no cookies, so credentials stay off.
43     app.add_middleware(
44         CORSMiddleware,
45         allow_origins=["*"],
46         allow_credentials=False,
47         allow_methods=["*"],
48         allow_headers=["*"],
49     )
50
51     # Serves static frontend files directly (now under api/ per approved structure)
52     os.makedirs("backend/api/static", exist_ok=True)
53     app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")
54
55     @app.get("/", response_class=HTMLResponse)
56     def serve_index():
57         index_path = "backend/api/static/index.html"
58         if os.path.exists(index_path):
59             with open(index_path, "r", encoding="utf-8") as f:
60                 return f.read()
61         return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"
62
63     # Global variables initialized at startup
64     db_instance: Optional[Database] = None
65     global_config: Optional[AppConfig] = None # now typed (Pydantic model)
66
67     # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 on purpose: a single
68     # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
69     # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
70     # already parallelize their AI handoff internally via their own pools.
71     _job_executor: Optional[ThreadPoolExecutor] = None
72
73     def _get_executor() -> ThreadPoolExecutor:
74         """Lazy module-global executor; tests monkeypatch this for inline execution."""
75         global _job_executor
76         if _job_executor is None:
77             _job_executor = ThreadPoolExecutor(max_workers=1,
thread_name_prefix="jobworker")
78         return _job_executor
79
80     # Legacy thin wrapper for any remaining direct calls during transition.
81     # New code should import load_app_config / AppConfig from backend.config directly.
82     def load_config(config_path: str = "config.json") -> Dict[str, Any]:
83         cfg = load_app_config(config_path)
84         return cfg.model_dump(mode="json")
85
86     # --- API Models ---
87     class ProcessRequest(BaseModel):
88         video_path: str
89         player_pool: Optional[List[str]] = None

```

```

90     max_frames: Optional[int] = None
91     segmenter: Optional[str] = None
92
93     class QueryRequest(BaseModel):
94         video_id: str
95         server: Optional[str] = None
96         receiver: Optional[str] = None
97         duration_min: Optional[float] = None
98         event_type: Optional[str] = None
99
100     class CompileRequest(BaseModel):
101         video_id: str
102         segment_ids: List[int]
103         output_filename: str
104
105     def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
106         """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
107
108         On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
109         fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
110         and
111         keep the prior good results intact ? a failed/empty re-run never destroys prior
112         data.
113         """
114         if segments:
115             db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
116         else:
117             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
118
119     # --- API Endpoints ---
120     @app.get("/api/config")
121     def get_config():
122         return global_config
123
124     def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
125         """The full hash ? validate ? segment ? report pipeline for one video.
126
127         This is the former synchronous /api/process body, unchanged in behavior, now run on
128         the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
129         the sync endpoint used to return (UI compatibility); the per-video observability
130         report is still emitted in `finally:` and grafted as response["reports"].
131
132         `progress` is an optional callable(stage: str) used to surface coarse job progress.
133         Raises on unexpected internal errors ? the caller records those as a job failure
134         (after this function has already restored the pre-run segments, see below).
135         """
136         notify = progress or (lambda stage: None)
137
138         notify("hashing")
139         video_id = compute_video_id(req.video_path)
140         was_reingest = db_instance.get_video(video_id) is not None
141
142         # 1. Run pluggable validators (with-context variant surfaces ffmpeg_meta for the
143         report)
144         notify("validating")
145         logger.info(f"Running validations for {req.video_path}...")
146         val_results, val_context = registry.run_all_with_context(req.video_path,
147         global_config)
148         failures = [r for r in val_results if not r.passed]
149
150         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
151         default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
152         "motion")
153         seg_name = req.segmenter or default_seg

```

```

150     segmenter_actual = None
151     segmentation_ran = False
152     response: Optional[Dict[str, Any]] = None
153     try:
154         if failures:
155             # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
156             # status; it no longer destroys the video's prior good segments.
157             db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
158             response = {
159                 "video_id": video_id,
160                 "status": "failed",
161                 "message": "Video failed validation checks.",
162                 "results": [r.model_dump() for r in val_results],
163             }
164             return response
165
166         # 2. Run segmenter & indexer
167         logger.info("[API] Video passed checks. Segmenting and indexing...")
168         db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])
169
170         segmenter_cls = segmenter_registry.get(seg_name)
171         if not segmenter_cls:
172             # Submit-time validation already 400s unknown names; this is the defensive
173             # in-worker path (e.g. config default pointing at an unregistered
segmenter).
174             available = list(segmenter_registry.list_available().keys())
175             response = {
176                 "video_id": video_id,
177                 "status": "failed",
178                 "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
179                 "results": [r.model_dump() for r in val_results],
180                 "play_segments": [],
181             }
182             return response
183
184         logger.info(f"[API] Using segmenter: {seg_name}")
185         notify(f"segmenting ({seg_name})")
186         segmenter_actual = seg_name
187         # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
188         # if the run yields segments; otherwise drop the new partial rows and keep the
old.
189         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
190         segmenter = segmenter_cls(db_instance, global_config)
191         try:
192             result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
193         except Exception:
194             # A raising segmenter must not leave half-written rows: restore the pre-run
195             # state (same destroy-after-confirm contract as the Failure branch), then
let
196             # the job worker record the failure.
197             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
198             raise
199         segmentation_ran = True
200
201         if isinstance(result, Failure):
202             logger.error(f"[API] Segmenter failed: {result.message}")
203             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
204             response = {
205                 "video_id": video_id,
206                 "status": "failed",

```

```

207         "message": f"Segmenter '{seg_name}' failed: {result.message}",
208         "results": [r.model_dump() for r in val_results],
209         "play_segments": [],
210     }
211     return response
212
213     segments = result.value if hasattr(result, "value") else result
214     notify("finalizing")
215     _finalize_reingest(video_id, segments, play_wm, cand_wm)
216
217     response = {
218         "video_id": video_id,
219         "status": "success",
220         "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
221         "results": [r.model_dump() for r in val_results],
222         "play_segments": segments,
223     }
224     return response
225 finally:
226     # Always emit the per-video observability report (next to the video, or to
227     # report.output_dir). Never raises. Surface the paths in the response.
228     paths = emit_indexer_report(
229         db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
230         val_results=val_results, val_context=val_context,
231         segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
232         was_reingest=was_reingest, segmentation_ran=segmentation_ran,
233     )
234     if paths and isinstance(response, dict):
235         response["reports"] = paths
236
237
238     class JobWaiting(Exception):
239         """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
240         WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`",
241         NOT a failure. The worker persists `next_poll_at` + `progress` (via
`set_job_waiting`),
242         releases the thread, and re-claims the job when due (`claim_due_job`). No work to
date
243         is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
contract.
244
245         The wiring is additive: the production segmenter path never raises this today (zero
behaviour change), so a job runs exactly as before. The hook is what a later slice
246         (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
247         """
248
249
250         def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
251             super().__init__(f"job parked for {delay_sec:.0f}s")
252             self.delay_sec = max(0.0, float(delay_sec))
253             self.progress = progress
254
255
256     def _job_to_request(job: Dict[str, Any]) -> ProcessRequest:
257         """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
258         the original submit. The row stores exactly the ProcessRequest fields."""
259         return ProcessRequest(
260             video_path=job["video_path"],
261             player_pool=job.get("player_pool"),
262             max_frames=job.get("max_frames"),
263             segmenter=job.get("segmenter"),
264         )
265
266

```

```

267
268 def _run_claimed_job(job: Dict[str, Any]) -> None:
269     """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
its
270     terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).
271
272     Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
means
273     the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
274     result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
the
275     job self-scheduled and will be re-claimed when `next_poll_at` is due.
276     """
277     job_id = job["id"]
278     req = _job_to_request(job)
279     try:
280         result = _execute_processing(
281             req, progress=lambda stage: db_instance.set_job_progress(job_id, stage))
282         if result.get("video_id"):
283             db_instance.set_job_video_id(job_id, result["video_id"])
284             db_instance.mark_job_done(job_id, result)
285     except JobWaiting as w:
286         # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
287         logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
288         db_instance.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
289     except Exception as e:
290         logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
291         db_instance.mark_job_failed(job_id, f"{type(e).__name__}: {e}")
292
293
294 def _drain_due_jobs() -> int:
295     """One worker tick: claim and run every currently-runnable job (queued, or waiting
whose
296     `next_poll_at` is due), earliest-due first, until the table has nothing runnable.
297
298     Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is
picked
299     up by a FUTURE drain once due; we schedule that follow-up drain via
`_schedule_drain` so
300     a single-worker box keeps making progress with no central timer (the DB is the
clock).
301     Returns the number of jobs that reached a terminal/parked state this tick.
302     """
303     ran = 0
304     while True:
305         job = db_instance.claim_due_job()
306         if job is None:
307             break
308         ran += 1
309         _run_claimed_job(job)
310     # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a
new
311     # submit. On the single-box executor this is a delayed re-drain; multi-worker just
relies
312     # on the next submit/drain. Best-effort ? never raises into the worker.
313     _schedule_next_drain_if_waiting()
314     return ran
315
316
317 def _schedule_next_drain_if_waiting() -> None:
318     """If any job is parked in 'waiting', arm a re-drain for when the soonest one is
due, so a
319     self-scheduled job resumes with no further client submit. The drain itself re-checks
320     due-ness, so an early wake is harmless and a clamped wake for a far-future job is
fine."""

```

```

321     try:
322         delay = db_instance.seconds_until_next_due()
323     except Exception as e: # pragma: no cover - defensive; never wedge the worker
324         logger.error(f"Could not compute next-due delay: {e}")
325         return
326     if delay is None:
327         return # nothing waiting
328     # Clamp so a far-future job doesn't pin a thread forever; the drain re-checks due-
329     _arm_wake_timer(max(0.0, min(delay, _MAX_DRAIN_WAKE_SEC)))
330
331
332     def _arm_wake_timer(delay_sec: float) -> None:
333         """Arm a single daemon timer that re-submits a drain after ``delay_sec`` (no central
334         scheduler ? the DB is the clock). Isolated as the sole timer-creating seam so tests
335         can
336         monkeypatch it to a no-op and assert the *scheduling decision* without real
337         waiting."""
338         timer = threading.Timer(delay_sec, lambda: _get_executor().submit(_drain_due_jobs))
339         timer.daemon = True
340         timer.start()
341
342     #: A parked job further out than this is re-checked by a capped wake rather than one
343     long
344     #: sleeping timer ? keeps the wake cadence bounded without a central scheduler.
345     _MAX_DRAIN_WAKE_SEC = 60.0
346
347     @app.post("/api/process", status_code=202)
348     def process_video(req: ProcessRequest):
349         """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
350
351         Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
352         client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
353         runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
354         worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
355         """
356         allowed_root = getattr(getattr(global_config, "security", None),
357         "allowed_video_root", None)
358         try:
359             validate_input_path(req.video_path, allowed_root=allowed_root)
360         except ValueError as e:
361             raise HTTPException(status_code=400, detail=str(e)) from e
362         if not os.path.exists(req.video_path):
363             raise HTTPException(status_code=404, detail=f"Video file not found at
364             '{req.video_path}'")
365         if req.segmenter and not segmenter_registry.get(req.segmenter):
366             available = list(segmenter_registry.list_available().keys())
367             raise HTTPException(
368                 status_code=400,
369                 detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
370                 {available}"
371             )
372         job_id = uuid.uuid4().hex
373         if not db_instance.create_job(job_id, req.video_path, req.segmenter,
374         req.player_pool, req.max_frames):
375             raise HTTPException(status_code=500, detail="Failed to persist job record.")
376         _get_executor().submit(_drain_due_jobs)
377         return JSONResponse(
378             status_code=202,
379             content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
380         )

```



```

379
380 @app.get("/api/jobs/{job_id}")
381 def get_job(job_id: str):
382     """Job state: {status, progress, result, reports}. `status` = execution state
383     (queued|running|done|failed); the pipeline verdict is result["status"]."""
384     job = db_instance.get_job(job_id)
385     if not job:
386         raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
387     result = job.get("result")
388     return {
389         "job_id": job["id"],
390         "status": job["status"],
391         "progress": job.get("progress"),
392         "video_id": job.get("video_id"),
393         "error": job.get("error"),
394         "result": result,
395         "reports": (result or {}).get("reports"),
396         "created_at": job.get("created_at"),
397         "started_at": job.get("started_at"),
398         "finished_at": job.get("finished_at"),
399     }
400
401 # --- Chunked upload (B2, PR-2) -----
402 # Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
403 # proxies cap request bodies at ~100 MB ? HOSTING_PLAN $2), are assembled server-side
404 # under security.upload_dir, then submitted to /api/process by the returned path.
405 # Upload state is in-process by design (this is the single-box alpha): a server restart
406 # aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
407 # arrives with PR-3 identity scoping.
408
409 _uploads: Dict[str, Dict[str, Any]] = {}
410 _uploads_lock = threading.Lock()
411
412
413 class UploadInitRequest(BaseModel):
414     filename: str
415     total_size_bytes: Optional[int] = None
416
417
418 class UploadCompleteRequest(BaseModel):
419     total_parts: int
420
421
422 def _upload_cfg():
423     sec = getattr(global_config, "security", None)
424     upload_dir = getattr(sec, "upload_dir", None) or "output/uploads"
425     max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
426     part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
427     exts = [e.lower().lstrip(".") for e in (getattr(sec, "allowed_upload_extensions",
428 None) or ["mp4", "mov"])]
429     return upload_dir, max_bytes, part_bytes, exts
430
431
432 def _abort_upload(upload_id: str) -> None:
433     with _uploads_lock:
434         st = _uploads.pop(upload_id, None)
435         if st:
436             try:
437                 os.remove(st["tmp_path"])
438             except OSError:
439                 pass
440
441
442 @app.post("/api/upload/init")
443 def upload_init(req: UploadInitRequest):

```

```

443     """Start a chunked upload. Returns upload_id; PUT sequential parts next."""
444     upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
445     try:
446         name = safe_output_filename(req.filename)
447     except ValueError as e:
448         raise HTTPException(status_code=400, detail=str(e)) from e
449     ext = os.path.splitext(name)[1].lower().lstrip(".")
450     if ext not in exts:
451         raise HTTPException(status_code=400,
452                             detail=f"Extension '{ext}' not allowed. Allowed:
{sorted(exts)}")
453     if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
454         raise HTTPException(status_code=413,
455                             detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB
upload limit.")
456     os.makedirs(upload_dir, exist_ok=True)
457     upload_id = uuid.uuid4().hex
458     tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
459     open(tmp_path, "wb").close()
460     with _uploads_lock:
461         _uploads[upload_id] = {"filename": name, "tmp_path": tmp_path, "parts": 0,
"bytes": 0}
462     return {"upload_id": upload_id, "max_part_mb": part_bytes // (1024 * 1024),
463           "next_part": f"/api/upload/{upload_id}/part/0"}
464
465
466 @app.put("/api/upload/{upload_id}/part/{index}")
467 async def upload_part(upload_id: str, index: int, request: Request):
468     """Append one part (raw bytes body). Parts are strictly sequential (0,1,2,...)."""
469     _, max_bytes, part_bytes, _ = _upload_cfg()
470     with _uploads_lock:
471         st = _uploads.get(upload_id)
472         if st is None:
473             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
474         if index != st["parts"]:
475             raise HTTPException(status_code=409,
476                                 detail=f"Out-of-order part: expected {st['parts']}, got
{index} (parts are sequential)")
477
478         received = 0
479         overflow = None
480         with open(st["tmp_path"], "ab") as fh:
481             async for chunk in request.stream():
482                 received += len(chunk)
483                 if received > part_bytes:
484                     overflow = f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part
limit."
485                     break
486                 if st["bytes"] + received > max_bytes:
487                     overflow = f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total
limit."
488                     break
489                 fh.write(chunk)
490         if overflow:
491             _abort_upload(upload_id)
492             raise HTTPException(status_code=413, detail=overflow + " Upload aborted ? re-
init to retry.")
493
494         with _uploads_lock:
495             st["parts"] += 1
496             st["bytes"] += received
497         return {"upload_id": upload_id, "received_parts": st["parts"], "received_bytes":
st["bytes"]}
498
499

```

```

500     @app.post("/api/upload/{upload_id}/complete")
501     def upload_complete(upload_id: str, req: UploadCompleteRequest):
502         """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
503         upload_dir, _, _, _ = _upload_cfg()
504         with _uploads_lock:
505             st = _uploads.get(upload_id)
506             if st is None:
507                 raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
508             if st["parts"] == 0 or st["parts"] != req.total_parts:
509                 raise HTTPException(status_code=409,
510                                     detail=f"Expected {req.total_parts} part(s), received
{st['parts']}.")
511             final_path = os.path.join(upload_dir, st["filename"])
512             if os.path.exists(final_path): # never overwrite an earlier upload with the same
name
513                 stem, ext = os.path.splitext(st["filename"])
514                 final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
515             os.replace(st["tmp_path"], final_path)
516             with _uploads_lock:
517                 _uploads.pop(upload_id, None)
518             video_id = compute_video_id(final_path)
519             return {"path": os.path.abspath(final_path), "video_id": video_id,
520                   "size_bytes": os.path.getsize(final_path),
521                   "next": "POST /api/process with this path"}
522
523
524     @app.delete("/api/upload/{upload_id}")
525     def upload_abort(upload_id: str):
526         """Abort an in-flight upload and delete its partial file."""
527         with _uploads_lock:
528             known = upload_id in _uploads
529             if not known:
530                 raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
531             _abort_upload(upload_id)
532             return {"status": "aborted", "upload_id": upload_id}
533
534
535     # --- Highlight download (B3, PR-2) -----
536
537     @app.get("/api/highlights/{video_id}/{filename}")
538     def download_highlight(video_id: str, filename: str):
539         """Stream a compiled highlight reel ? `filename` is the bare name given to
/api/compile
540         (which only writes into output_dir; the browser previously got back an unreachable
541         server-local path)."""
542         try:
543             name = safe_output_filename(filename)
544         except ValueError as e:
545             raise HTTPException(status_code=400, detail=str(e)) from e
546         if not db_instance.get_video(video_id):
547             raise HTTPException(status_code=404, detail="Indexed video record not found.")
548         output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
549         or "output"
550         path = os.path.join(output_dir, name)
551         try:
552             path = resolve_under_root(path, output_dir)
553         except ValueError as e:
554             raise HTTPException(status_code=400, detail=str(e)) from e
555         if not os.path.isfile(path):
556             raise HTTPException(status_code=404,
557                                 detail=f"No compiled file named '{name}'. Compile first via
/api/compile.")
558         media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
559         return FileResponse(path, media_type=media, filename=name)

```

```

559
560
561 @app.post("/api/query")
562 def query_play_segments(req: QueryRequest):
563     filters = {
564         "server": req.server,
565         "receiver": req.receiver,
566         "duration_min": req.duration_min,
567         "event_type": req.event_type
568     }
569     segments = db_instance.query_play_segments(req.video_id, filters)
570     return {"play_segments": segments}
571
572 @app.post("/api/compile")
573 def compile_highlights(req: CompileRequest):
574     video = db_instance.get_video(req.video_id)
575     if not video:
576         raise HTTPException(status_code=404, detail="Indexed video record not found.")
577
578     # Retrieve matching play segments from db
579     all_segments = db_instance.query_play_segments(req.video_id, {})
580     matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]
581
582     if not matched_segments:
583         raise HTTPException(status_code=400, detail="No matching play segments found to
584 stitch.")
585
586     # Reject path traversal / absolute output names (os.path.join would otherwise let an
587     # absolute or '..'-laden output_filename write anywhere on disk).
588     try:
589         out_name = safe_output_filename(req.output_filename)
590     except ValueError as e:
591         raise HTTPException(status_code=400, detail=str(e)) from e
592
593     # The configured shot-count floor (stitching.min_shot_count) applies on the API
594     # path too ? parity with scripts/run_process_and_stitch.py. Segments WITHOUT
595     shot_count
596     # metadata are exempt: the floor filters known counts only, so explicitly
597     # requested manual/legacy segments can't silently vanish under the default floor.
598     stitching = getattr(global_config, "stitching", None) or StitchingConfig()
599     kept = []
600     for s in matched_segments:
601         meta = s.get("metadata") or {}
602         sc = meta.get("shot_count") if isinstance(meta, dict) else None
603         try:
604             if sc is not None and int(sc) < stitching.min_shot_count:
605                 continue
606             except (TypeError, ValueError):
607                 pass # unparseable count -> treat as unknown, keep
608             kept.append(s)
609         if len(kept) < len(matched_segments):
610             logger.info(f"[compile] stitching.min_shot_count={stitching.min_shot_count}:
611 dropped "
612 f"{len(matched_segments) - len(kept)}/{len(matched_segments)}
613 segments below the floor.")
614         if not kept:
615             raise HTTPException(status_code=400,
616 detail=f"All {len(matched_segments)} matching segments fall
617 below "
618 f"stitching.min_shot_count={stitching.min_shot_count}.")
619
620     # Extract start/end time pairs
621     time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
622

```

```

618         # Run compiler with the configured stitching paddings + optional branding watermark
619         output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
620         compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
621                                 end_padding=stitching.end_padding,
watermark=stitching.watermark)
622         try:
623             out_path = compiler.compile_highlights(video["filepath"], time_pairs, out_name)
624             return {"status": "success", "filepath": out_path}
625         except Exception as e:
626             raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}") from e
627
628     # --- CLI Debug Utility & Orchestration ---
629     def run_cli_diagnose_clip(video_path: str, timestamp: float):
630         """CLI debugging utility to examine segment properties around a timestamp."""
631         print("-" * 60)
632         print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
633         print("-" * 60)
634
635         cfg = load_config() # legacy wrapper still works, returns dict for now
636
637         # 1. Validation check diagnostics
638         print("[DIAGNOSTIC] Running validation framework...")
639         results = registry.run_all(video_path, cfg)
640         for r in results:
641             status_symbol = "[PASS]" if r.passed else "[FAIL]"
642             print(f"    {status_symbol} {r.validator_name}: {r.message}")
643             if r.details:
644                 print(f"        Details: {r.details}")
645
646         # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
647         print("-" * 60)
648         print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
649         import cv2
650         cap = cv2.VideoCapture(video_path)
651         if not cap.isOpened():
652             print("[FAIL] OpenCV could not open video.")
653             return
654
655         fps = cap.get(cv2.CAP_PROP_FPS)
656         total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)
657
658         start_frame = max(0, int((timestamp - 3.0) * fps))
659         end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))
660
661         # Measure frame-by-frame changes in sample region
662         cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
663         prev_gray = None
664         motions = []
665
666         for _ in range(start_frame, end_frame):
667             ret, frame = cap.read()
668             if not ret:
669                 break
670             gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
671             gray = cv2.GaussianBlur(gray, (21, 21), 0)
672
673             if prev_gray is not None:
674                 diff = cv2.absdiff(prev_gray, gray)
675                 _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
676                 motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
677                 motions.append(motion_ratio)
678             prev_gray = gray

```

```

679
680     cap.release()
681
682     if motions:
683         avg_motion = sum(motions) / len(motions)
684         # Support both legacy dict (from wrapper) and future direct model
685         if hasattr(cfg, "indexing"):
686             threshold = getattr(cfg.indexing, "motion_threshold", 0.08)
687         else:
688             threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08) # type:
ignore[attr-defined]
689         print(f" Sample Frame Count: {len(motions)}")
690         print(f" Average motion index around timestamp: {round(avg_motion, 4)}
(threshold: {threshold})")
691         if avg_motion > threshold:
692             print(" Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play.")
693         else:
694             print(" Result: [DEAD TIME] Idle movement. Segment classified as dead
time.")
695     else:
696         print(" [ERROR] No visual frames were extracted in the sample range.")
697
698     print("=" * 60)
699
700     def main():
701         parser = argparse.ArgumentParser(description="Sports Highlight Indexer
Orchestrator")
702         parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
703         parser.add_argument("--output-dir", default="output", help="Directory where
processed clips/highlights are saved")
704         parser.add_argument("--setup", action="store_true", help="Launch dependency checker
and setup utility")
705         parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a
video clip to run detailed diagnostics on")
706         parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
707         parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
708         parser.add_argument("--max-frames", type=int, help="Limit processing to the first N
sub-sampled frames for debugging")
709
710         available_segmenters = list(segmenter_registry.list_available().keys())
711         parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
712         parser.add_argument("--trim-start", type=float, help="Start time in seconds for the
debug trim segment feature")
713         parser.add_argument("--trim-end", type=float, help="End time in seconds for the
debug trim segment feature")
714         parser.add_argument("--trim-output", help="Output filename for the debug trimmed
segment (saves in output-dir unless absolute path)")
715
716         args = parser.parse_args()
717
718         if args.setup:
719             # Invoke the diagnostics/bootstrap script (renamed from the old root
720             # setup.py to scripts/doctor.py so it no longer shadows the PEP 517
721             # build system). Resolve its path from this file so `indexer --setup`
722             # works regardless of the caller's cwd.
723             import subprocess
724             doctor_path = os.path.join(
725                 os.path.dirname(os.path.abspath(__file__)),
726                 "scripts", "doctor.py",
727             )
728             print(f"[INFO] Invoking diagnostics: {doctor_path} ...")

```

```

729         subprocess.run([sys.executable, doctor_path])
730         return
731
732     if args.trim_start is not None and args.trim_end is not None:
733         if not args.video_path:
734             print("[ERROR] Please provide --video-path to extract a segment.")
735             return
736
737         # Determine output path
738         out_filename = args.trim_output or "trimmed_segment.mp4"
739         if os.path.isabs(out_filename):
740             out_path = out_filename
741             out_dir = os.path.dirname(out_path)
742             out_name = os.path.basename(out_path)
743         else:
744             out_dir = args.output_dir
745             out_path = os.path.join(out_dir, out_filename)
746             out_name = out_filename
747
748         os.makedirs(out_dir, exist_ok=True)
749         print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")
750
751         # global_config isn't initialized this early; load the typed config so the trim
752         # clip carries the same configured stitching paddings + watermark as a real
753         # highlight (watermark default-on).
754         stitching = load_app_config().stitching
755         compiler = VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
756                                 end_padding=stitching.end_padding,
watermark=stitching.watermark)
757         try:
758             res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
759             print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
760         except Exception as e:
761             print(f"[ERROR] Failed to compile trimmed segment: {e}")
762         return
763
764     if args.debug_clip is not None:
765         if not args.video_path:
766             print("[ERROR] Please provide --video-path to debug a specific clip.")
767             return
768         run_cli_diagnose_clip(args.video_path, args.debug_clip)
769         return
770
771     # Initialize Global Config and DB
772     global global_config, db_instance
773     global_config = load_app_config() # returns typed AppConfig
774
775     # The CLI --output-dir overrides the configured output directory. It now lives
776     # on the typed model (OutputConfig.output_dir), so every consumer ? including
777     # /api/compile ? reads the same value instead of a write-only runtime hack.
778     global_config.output.output_dir = args.output_dir
779     os.makedirs(global_config.output.output_dir, exist_ok=True)
780
781     db_path = os.path.join(global_config.output.output_dir,
global_config.output.db_name)
782     db_instance = Database(db_path)
783
784     # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
785     # the executor is in-process. Mark them failed so pollers see a terminal state.
786     swept = db_instance.fail_stale_jobs()
787     if swept:
788         logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
failed.")

```

```

789
790     # CLI processing mode (no web UI needed)
791     if args.video_path and not args.server:
792         print(f"[CLI] Processing video file: {args.video_path}")
793         if not os.path.exists(args.video_path):
794             print(f"[FAIL] Video file not found: {args.video_path}")
795             return
796
797         video_id = compute_video_id(args.video_path)
798         was_reingest = db_instance.get_video(video_id) is not None
799
800         # Validate (with-context variant surfaces ffprobe_meta for the report)
801         print("[CLI] Validating...")
802         val_results, val_context = registry.run_all_with_context(args.video_path,
global_config)
803         failures = [r for r in val_results if not r.passed]
804
805         # Save video status
806         status = "failed" if failures else "processed"
807         db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r
in val_results])
808
809         seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
810         segmenter_actual = None
811         segmentation_ran = False
812         try:
813             print("\n" + "="*40)
814             print("VALIDATION SUMMARY")
815             print("="*40)
816             for r in val_results:
817                 status_symbol = "[PASS]" if r.passed else "[FAIL]"
818                 print(f"{status_symbol} {r.validator_name}: {r.message}")
819             print("="*40 + "\n")
820
821             if failures:
822                 print("[FAIL] Video failed checks. Indexing halted.")
823                 return
824
825             # Index
826             segmenter_cls = segmenter_registry.get(seg_name)
827             if not segmenter_cls:
828                 print(f"[FAIL] Segmenter '{seg_name}' not found.")
829                 return
830
831             print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
832             segmenter_actual = seg_name
833             play_wm, cand_wm = db_instance.segment_watermarks(video_id)
834             segmenter = segmenter_cls(db_instance, global_config)
835             result = segmenter.process_video(video_id, args.video_path,
max_frames=args.max_frames)
836             segmentation_ran = True
837
838             if isinstance(result, Failure):
839                 print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
840                 print("[FAIL] Indexing halted for video.")
841                 segments = []
842             else:
843                 segments = result.value if hasattr(result, "value") else result
844                 print(f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}")
845                 # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
846                 _finalize_reingest(video_id, segments, play_wm, cand_wm)
847             finally:

```



```

848         # Always emit the per-video observability report. Never raises.
849         paths = emit_indexer_report(
850             db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
851             val_results=val_results, val_context=val_context,
852             segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
853             was_reingest=was_reingest, segmentation_ran=segmentation_ran,
854         )
855         if paths:
856             print(f"[REPORT] Technical report : {paths.get('technical_md')}")
857             print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
858             print(f"[REPORT] Machine JSON      : {paths.get('json')}")
859         return
860
861     # Start Local Server Mode
862     if args.server or not args.video_path:
863         print(f"[SERVER] Starting local FastAPI server on
http://localhost:{args.port}...")
864         # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3):
the
865         # in-process wake-timer for any parked 'waiting' job died with the old process,
so
866         # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
867         _get_executor().submit(_drain_due_jobs)
868         uvicorn.run(app, host="127.0.0.1", port=args.port)
869
870     if __name__ == "__main__":
871         main()
872

```

9. assistant (2026-06-18T01:33:27.638Z)

This is a rich target. Let me examine the upload part handler carefully ? there's a potential issue with the overflow/byte accounting. Let me also look at the CLI argparse files and validation helpers.

10. user (2026-06-18T01:33:29.054Z)

```

1      """Path-safety helpers for the API boundary (untrusted request fields)."""
2      import os
3      from typing import Optional
4
5
6      def safe_output_filename(name: str) -> str:
7          """Validate an output filename is a BARE name (no dir part, no traversal, not
absolute).
8
9          `os.path.join(output_dir, name)` silently discards `output_dir` when `name` is
absolute,
10         and `..` segments escape it ? so an unchecked `output_filename` is an arbitrary-
write
11         primitive. Returns the name unchanged when safe; raises ValueError otherwise.
12         """
13         if not name or not name.strip():
14             raise ValueError("output_filename must not be empty")
15         if "\x00" in name:
16             raise ValueError("output_filename must not contain a null byte")
17         if "/" in name or "\\" in name:
18             raise ValueError(f"output_filename must not contain a path separator: {name!r}")
19         if os.path.isabs(name) or name != os.path.basename(name) or name in (".", ".."):
20             raise ValueError(f"output_filename must be a bare filename, got: {name!r}")
21         return name
22
23
24     def resolve_under_root(path: str, root: str) -> str:

```

```

25         """Return the real path, asserting it is contained under `root` (raises ValueError
if not)."""
26         rp = os.path.realpath(path)
27         rr = os.path.realpath(root)
28         try:
29             contained = os.path.commonpath([rp, rr]) == rr
30         except ValueError: # different drives on Windows
31             contained = False
32         if not contained:
33             raise ValueError(f"path {path!r} escapes the allowed root {root!r}")
34         return rp
35
36
37     def validate_input_path(path: str, allowed_root: Optional[str] = None) -> None:
38         """Reject null bytes and (when an allowed_root is configured) out-of-root inputs.
39
40         With no allowed_root (default, single-user local mode) only the null-byte check
applies,
41         so the local 'point it at any of my videos' workflow is preserved. Setting an
42         allowed_root (a future hosted/tenant deployment) enforces containment.
43         """
44         if "\x00" in path:
45             raise ValueError("video_path must not contain a null byte")
46         if allowed_root:
47             resolve_under_root(path, allowed_root)
48

```

11. user (2026-06-18T01:33:29.227Z)

```

1     """CLI to evaluate detected rallies against hand-annotated ground truth.
2
3     Scores predictions already stored in the SQLite DB (run the pipeline first) ? no
4     API calls, fully deterministic. See docs/EVALUATION.md for the workflow.
5
6     Examples
7     -----
8     Create an empty annotation file to fill in:
9         python run_eval.py --scaffold --annotations rallies.csv
10
11     Score a video's detected rallies (identify it by file or by stored video_id):
12         python run_eval.py --video "C:/path/match.mp4" --annotations rallies.csv
13         python run_eval.py --video-id <md5> --annotations rallies.csv --stage both --iou 0.5
14
15     Show which rallies were missed / spurious, and dump a JSON report:
16         python run_eval.py --video-id <md5> --annotations rallies.csv --show-failures --json
report.json
17     """
18
19     import os
20     import sys
21     import json
22     import argparse
23     import logging
24
25     from backend.infrastructure.database import Database
26     from backend.config import load_config
27     from backend.utils.hashing import compute_video_id as get_file_md5 # canonical video_id
28     from backend.eval.annotations import load_annotations, write_scaffold
29     from backend.eval.harness import evaluate, format_report_text, report_to_dict, STAGES
30
31
32     def _default_db_path() -> str:
33         """Resolve the DB path from the typed config (output.output_dir /
output.db_name)."""
34         cfg = load_config()

```

```

35         return os.path.join(cfg.output.output_dir, cfg.output.db_name)
36
37
38     _EPILOG = """\
39 Prerequisite:
40     The harness scores predictions read from the SQLite DB ? it does NOT run the
41     pipeline. So you must first PROCESS the video (e.g. via run_process_and_stitch.py)
42     so its detections land in output/sports_indexer.db. Then point this tool at the
43     same video file (--video) or its stored MD5 (--video-id). If you get
44     "database not found" or zero predictions, the video hasn't been processed yet.
45
46     See docs/EVALUATION.md for the annotation format and how to read the output.
47     """
48
49
50     def build_parser() -> argparse.ArgumentParser:
51         p = argparse.ArgumentParser(
52             description="Evaluate detected rallies vs ground-truth annotations.",
53             epilog=_EPILOG,
54             formatter_class=argparse.RawDescriptionHelpFormatter,
55         )
56         p.add_argument("--annotations", help="Path to the ground-truth rally CSV (start,end
57 per row).")
58         p.add_argument("--video", help="Path to the video file (its MD5 becomes the
59 video_id).")
60         p.add_argument("--video-id", help="Stored video_id (MD5) to evaluate, instead of
61 --video.")
62         p.add_argument("--db", default=None, help="SQLite DB path (default: from
63 config.json).")
64         p.add_argument("--stage", choices=("candidates", "final", "both"), default="both",
65             help="Which prediction stage(s) to score (default: both).")
66         p.add_argument("--detector", default=None,
67             help="Only score candidates from this detector (e.g. yolo_hybrid,
68 tracknet_hybrid).")
69         p.add_argument("--iou", type=float, default=0.5, help="IoU match threshold (default:
70 0.5).")
71         p.add_argument("--pr-curve", action="store_true", help="Include an IoU sweep
72 (0.1..0.9) in the report.")
73         p.add_argument("--show-failures", action="store_true",
74             help="List missed (FN) and spurious (FP) rallies with timestamps.")
75         p.add_argument("--json", dest="json_out", default=None, help="Write a JSON report to
76 this path.")
77         p.add_argument("--scaffold", action="store_true",
78             help="Write an empty annotation CSV to --annotations and exit.")
79         return p
80
81
82     def main(argv=None) -> int:
83         logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
84         args = build_parser().parse_args(argv)
85
86         # Scaffold mode: just write the template and exit.
87         if args.scaffold:
88             if not args.annotations:
89                 print("ERROR: --scaffold requires --annotations <path>", file=sys.stderr)
90                 return 2
91             write_scaffold(args.annotations)
92             print(f"Wrote empty annotation file: {args.annotations}")
93             return 0
94
95         if not args.annotations:
96             print("ERROR: --annotations is required (or use --scaffold).", file=sys.stderr)
97             return 2
98
99         # Resolve the video_id.

```

```

92     if args.video_id:
93         video_id = args.video_id
94     elif args.video:
95         if not os.path.exists(args.video):
96             print(f"ERROR: video not found: {args.video}", file=sys.stderr)
97             return 2
98         print("Computing video MD5...")
99         video_id = get_file_md5(args.video)
100        print(f"video_id = {video_id}")
101    else:
102        print("ERROR: provide --video <path> or --video-id <md5>.", file=sys.stderr)
103        return 2
104
105    db_path = args.db or _default_db_path()
106    if not os.path.exists(db_path):
107        print(f"ERROR: database not found at {db_path}. Run the pipeline first.",
108              file=sys.stderr)
109        return 2
110
111    gt = load_annotations(args.annotations)
112    if not gt:
113        print(f"WARNING: no ground-truth rallies in {args.annotations}.",
114              file=sys.stderr)
115
116    stages = list(STAGES) if args.stage == "both" else [args.stage]
117    db = Database(db_path)
118    report = evaluate(db, video_id, gt, stages, iou_threshold=args.iou,
119                    detector=args.detector, with_pr_curve=args.pr_curve)
120
121    print(format_report_text(report, show_failures=args.show_failures))
122
123    if args.json_out:
124        with open(args.json_out, "w") as f:
125            json.dump(report_to_dict(report), f, indent=2)
126            print(f"\nJSON report written to {args.json_out}")
127
128    return 0
129
130 if __name__ == "__main__":
131     raise SystemExit(main())

```

12. assistant (2026-06-18T01:33:35.131Z)

Let me look closely at the `upload_part` overflow logic. There's a subtle issue with the byte accounting. Let me trace through it carefully.

In `upload_part`, when overflow occurs mid-stream, chunks already written to disk before the overflow are NOT counted (`st["bytes"]` is only updated at the end if no overflow). But the upload is aborted in that case, so it's fine. Let me look at the case where `received > part_bytes` ? the check happens AFTER `received += len(chunk)` but the chunk is NOT written when it overflows (break before `fh.write`). That's fine.

Let me examine the other CLI tools and the database pagination/query methods.

13. user (2026-06-18T01:33:36.090Z)

```

1     """CLI: regression-test the production tool against golden-set ground truth.
2
3     Obtains ground truth from an annotation provider (a tier), scores the video's
4     stored predictions, and compares to a snapshotted baseline. Exit code 1 on a
5     detected regression so it can gate CI. See docs/GOLDEN_SET_IMPLEMENTATION.md.
6
7     Examples

```

```

8         -----
9         Snapshot the current numbers as the baseline (first run / after a real improvement):
10         python run_regression.py --video-id <md5> --tier file --annotations rallies.csv
--update-baseline
11
12         Check for a regression (exit 1 if precision/recall dropped beyond tolerance):
13         python run_regression.py --video-id <md5> --tier file --annotations rallies.csv
14         """
15         import os
16         import sys
17         import json
18         import argparse
19         import logging
20
21         from backend.infrastructure.database import Database
22         from backend.config import load_config
23         from backend.annotations import annotation_registry
24         from backend.eval import regression
25
26
27         def _default_db_path() -> str:
28             cfg = load_config()
29             return os.path.join(cfg.output.output_dir, cfg.output.db_name)
30
31
32         def build_parser() -> argparse.ArgumentParser:
33             p = argparse.ArgumentParser(description="Regression-test the tool vs golden-set
ground truth.")
34             p.add_argument("--video-id", required=True, help="Stored video_id (MD5) whose
predictions to score.")
35             p.add_argument("--tier", default="file",
36                             help=f"Annotation provider tier. Available:
{annotation_registry.list_available()}")
37             p.add_argument("--annotations", help="video_ref for the provider (e.g. CSV path for
the 'file' tier).")
38             p.add_argument("--stage", choices=("candidates", "final"), default="final")
39             p.add_argument("--detector", default=None, help="Restrict candidate scoring to one
detector.")
40             p.add_argument("--iou", type=float, default=0.5)
41             p.add_argument("--tolerance", type=float, default=regression.DEFAULT_TOLERANCE)
42             p.add_argument("--db", default=None)
43             p.add_argument("--baseline", default=regression.DEFAULT_BASELINE_PATH,
help="Baseline JSON path.")
44             p.add_argument("--update-baseline", action="store_true",
45                             help="Snapshot the current numbers as the new baseline instead of
checking.")
46             p.add_argument("--allow-missing-baseline", action="store_true",
47                             help="Treat a missing baseline as a pass (exit 0) instead of a gate
error (exit 3).")
48             p.add_argument("--json", dest="json_out", default=None)
49             return p
50
51
52         def main(argv=None) -> int:
53             logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
54             args = build_parser().parse_args(argv)
55
56             db_path = args.db or _default_db_path()
57             if not os.path.exists(db_path):
58                 print(f"ERROR: database not found at {db_path}. Run the pipeline first.",
file=sys.stderr)
59                 return 2
60             db = Database(db_path)
61
62             result = regression.regress_with_provider(

```

```

63         db, video_id=args.video_id, video_ref=args.annotations, tier=args.tier,
64         stage=args.stage, iou_threshold=args.iou, detector=args.detector,
65         tolerance=args.tolerance, baseline_path=args.baseline,
66     )
67
68     print(regression.format_result(result))
69
70     if args.update_baseline:
71         regression.save_baseline(result.video_id, result.stage, result.precision,
72                                 result.recall, result.f1, path=args.baseline,
73                                 iou_threshold=result.iou_threshold,
74                                 detector=args.detector,
75                                 gt_fingerprint=result.gt_hash)
76         print(f"Baseline updated in {args.baseline}.")
77
78     if args.json_out:
79         with open(args.json_out, "w") as f:
80             json.dump(result.as_dict(), f, indent=2)
81             print(f"JSON written to {args.json_out}")
82
83     # Exit codes: 1 = regression, 3 = no baseline to compare (gate cannot pass), 0 = ok.
84     if result.regressed:
85         return 1
86     if not result.has_baseline and not args.update_baseline and not
87     args.allow_missing_baseline:
88         print("ERROR: no baseline to compare against ? the gate cannot verify anything.
89         "
90             "Snapshot one with --update-baseline (then commit it), or pass "
91             "--allow-missing-baseline to accept this run.", file=sys.stderr)
92         return 3
93     return 0
94
95 if __name__ == "__main__":
96     raise SystemExit(main())

```

14. user (2026-06-18T01:33:36.678Z)

```

1     """Batch highlight-reel runner ? orchestrates "generate highlights for these videos
2     into this folder" requests with CLEAN, NON-DESTRUCTIVE, resumable execution.
3
4     Why this exists (lessons paid for the hard way):
5     - **Never delete other work.** Reels + per-video run.logs are written to a LOCAL work
6     dir
7     first, then COPIED into the destination (atomically: temp name -> os.replace). The
8     destination folder is only ever ADDED to ? we never bulk-delete it. (A bulk `rm` of a
9     Google-Drive-synced folder once lost finished reels + logs.)
10    - **Drive-sync safe.** Generating directly inside a Google Drive folder races the sync
11    client (deleted files resurrect, partial writes sync). Generate locally; publish once.
12    - **Idempotent re-runs.** Each video's prior DB segments are cleared before processing
13    (add_video is an UPSERT and does NOT clear children), so a re-run never accumulates
14    duplicate rallies. Belt-and-suspenders with VideoCompiler's overlap-merge (PR #180).
15    - **Resumable & concurrency-friendly.** A video whose reel already exists in the
16    destination is skipped (unless --force); launching a second request never undoes the
17    first's published reels. The Gemini path also isolates its chunk workdir per video
18    (PR for output/chunks/<video_id>) so concurrent videos don't collide.
19
20    Usage:
21    python -m tools.generate_highlights --videos-dir "G:\\...\\June 12 2026" \
22    --out "G:\\...\\June 12 2026\\Highlights" --segmenter gemini [--force]
23    python -m tools.generate_highlights --video A.MP4 --video B.MP4 --out <dir> ...
24    """
25    from __future__ import annotations

```

```

26     import argparse
27     import logging
28     import os
29     import shutil
30     import sys
31     import traceback
32     from typing import Any, Dict, List, Optional, cast
33
34     from dotenv import load_dotenv
35
36     sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
37
38     from backend.config import load_config
39     from backend.infrastructure.database import Database
40     from backend.pipeline.segmenters.base import segmenter_registry
41     from backend.pipeline.stitcher.compiler import VideoCompiler
42     from backend.results import Failure
43     from backend.utils.hashing import compute_video_id
44
45     logger = logging.getLogger("highlights_runner")
46
47     # Detectors that run in WSL can't read Google Drive / UNC via /mnt -> localize first.
48     _WSL_DETECTORS = {"wasb_hybrid", "tracknet_hybrid", "yolo_hybrid"}
49     _VIDEO_EXTS = (".mp4", ".MP4", ".mov", ".MOV", ".mkv", ".avi")
50
51
52     def _atomic_publish(local_path: str, dest_path: str) -> None:
53         """Copy local_path -> dest_path so the destination never sees a partial file.
54
55         Copy to a temp sibling, then os.replace (atomic on the same volume). Never deletes
56         anything else in the destination folder.
57         """
58         os.makedirs(os.path.dirname(dest_path), exist_ok=True)
59         tmp = dest_path + ".publishing.tmp"
60         shutil.copy2(local_path, tmp)
61         os.replace(tmp, dest_path)
62
63
64     def generate_highlights_batch(video_paths: List[str], out_dir: str, *, segmenter: str,
65                                  local_work_dir: str, db_path: str, force: bool = False,
66                                  skip_ai_handoff: bool = False, wasb_stream: bool = False,
67                                  detect_from_paths: Optional[List[str]] = None) -> dict:
68         """Generate one highlight reel per input video into out_dir. Returns a summary dict.
69
70         Non-destructive: only ADDS reels/logs to out_dir (atomic publish); never deletes.
71
72         skip_ai_handoff: run the trajectory-hybrid segmenters fully-local (no Gemini
73         handoff);
74         candidate windows become the rallies. wasb_stream: have the WASB detector decode
75         the
76         video on the fly instead of dumping ~46k PNGs (output-preserving fast path,
77         #178).
78         detect_from_paths: optional list parallel to video_paths. When given, the detector
79         runs on
80         detect_from_paths[i] (a cheaper proxy ? e.g. a 1080p downscale of a 4K source)
81         but the
82         reel is stitched from video_paths[i] at full resolution. Rally timestamps
83         transfer 1:1
84         (a pure temporal-preserving downscale shares the timeline). None = detect+stitch
85         the
86         same source (legacy behaviour).
87         """
88         os.makedirs(out_dir, exist_ok=True)
89         os.makedirs(local_work_dir, exist_ok=True)
90         needs_local = segmenter in _WSL_DETECTORS

```

```

84
85         # Load .env (e.g. GEMINI_API_KEY) so the Gemini handoff works when the runner is
invoked
86         # standalone ? backend.main does this, but this module is its own entrypoint.
Without it the
87         # AI segmenter finds no API key and silently produces zero rallies. The fully-local
path
88         # (skip_ai_handoff) needs no key, so a missing .env is harmless there.
89         load_dotenv()
90
91         cfg = load_config()
92         # Per-run overrides (typed config; the segmenter reads these via its dict-compat
.get()).
93         # Logged by the segmenter itself once logging is configured below.
94         if skip_ai_handoff:
95             cfg.indexing.skip_ai_handoff = True
96         if wasb_stream:
97             cfg.indexing.wasb.stream_video = True
98         db = Database(db_path)
99         seg_cls = segmenter_registry.get(segmenter)
100         if seg_cls is None:
101             raise SystemExit(f"unknown segmenter '{segmenter}'; available:
{segmenter_registry.list_available()}")
102
103         # Combined batch log (local; published at the end).
104         fmt = logging.Formatter("%(asctime)s [(levelname)s] %(name)s: %(message)s",
"%H:%M:%S")
105         if not logging.getLogger().handlers:
106             logging.basicConfig(level=logging.INFO,
handlers=[logging.StreamHandler(sys.stdout)])
107             for h in logging.getLogger().handlers:
108                 h.setFormatter(fmt)
109         batch_log_local = os.path.join(local_work_dir, "_batch_run.log")
110         batch_fh = logging.FileHandler(batch_log_local, mode="w", encoding="utf-8")
111         batch_fh.setFormatter(fmt)
112         logging.getLogger().addHandler(batch_fh)
113
114         summary: Dict[str, List[str]] = {"done": [], "skipped": [], "no_rallies": [],
"failed": []}
115         logger.info("highlights runner: %d videos -> %s (segmenter=%s, force=%s)",
116                     len(video_paths), out_dir, segmenter, force)
117
118         for i, src in enumerate(video_paths, 1):
119             name = os.path.basename(src)
120             stem = os.path.splitext(name)[0]
121             reel_name = f"{stem} - Highlights.mp4"
122             dest_reel = os.path.join(out_dir, reel_name)
123             local_reel = os.path.join(local_work_dir, reel_name)
124             # Detection source: a (cheaper) proxy when --detect-from is paired with --video;
125             # otherwise the video itself. The reel is ALWAYS stitched from `src` (full res).
126             detect_src = detect_from_paths[i - 1] if detect_from_paths else src
127
128             if not force and os.path.exists(dest_reel) and os.path.getsize(dest_reel) > 0:
129                 logger.info("[%d/%d] SKIP %s ? reel already published", i, len(video_paths),
name)
130                 summary["skipped"].append(name)
131                 continue
132             if not os.path.exists(src) or not os.path.exists(detect_src):
133                 logger.warning("[%d/%d] SKIP %s ? source missing (stitch_exists=%s
detect_exists=%s)",
134                                i, len(video_paths), name, os.path.exists(src),
os.path.exists(detect_src))
135                 summary["failed"].append(name)
136                 continue
137

```



```

138     run_log_local = os.path.join(local_work_dir, f"{stem} - run.log")
139     run_fh = logging.FileHandler(run_log_local, mode="w", encoding="utf-8")
140     run_fh.setFormatter(fmt)
141     logging.getLogger().addHandler(run_fh)
142     staged_src: Optional[str] = None
143     try:
144         logger.info("[%d/%d] === %s ===", i, len(video_paths), name)
145         # Localize the DETECTION source for WSL detectors (the WSL FS can't read
Drive/UNC
146         # via /mnt). The stitch source (`src`) is read by ffmpeg directly during
compile, so
147         # only the detector input ever needs staging.
148         if needs_local:
149             staged_src = os.path.join(local_work_dir, os.path.basename(detect_src))
150             if not (os.path.exists(staged_src) and os.path.getsize(staged_src) ==
os.path.getsize(detect_src)):
151                 logger.info("localizing detection source (%.0f MB)...",
os.path.getsize(detect_src) / 1024 / 1024)
152                 shutil.copy2(detect_src, staged_src)
153                 proc_detect = staged_src
154             else:
155                 proc_detect = detect_src
156
157         vid = compute_video_id(proc_detect)
158         db.clear_video_data(vid) # idempotent: no cross-run accumulation
159         db.add_video(vid, proc_detect, "processed", [])
160         if detect_src != src:
161             logger.info("detecting on proxy '%s' -> stitching reel from '%s'",
162                 os.path.basename(detect_src), name)
163             logger.info("video_id=%s segmenting (%s)...", vid, segmenter)
164
165         # AppConfig is consumed by segmenters via its dict-compatible .get shim (same as
166         # main.py); cast keeps the type checker honest about that contract.
167         result = seg_cls(db, cast(Dict[str, Any], cfg)).process_video(vid,
proc_detect)
168         if isinstance(result, Failure):
169             logger.error("segmentation FAILED: %s", getattr(result, "message",
result))
170             summary["failed"].append(name)
171             continue
172
173         segs = db.query_play_segments(vid, {})
174         pairs = sorted((float(s["start_time"]), float(s["end_time"])) for s in segs)
175         logger.info("rallies detected: %d", len(pairs))
176         if not pairs:
177             logger.warning("no rallies -> no reel")
178             summary["no_rallies"].append(name)
179             continue
180
181         # Stitch source: the original `src` when detecting on a separate proxy (hi-
res reel);
182         # otherwise the (possibly localized) detection input ? identical to legacy
behaviour.
183         stitch_src = src if detect_from_paths else proc_detect
184         # Honor the configured branding watermark + segment paddings (stitching.*).
Read
185         # dict-compatibly so a plain-dict config (tests) and the typed AppConfig
both work.
186         # NOTE: the runner previously dropped these ? reels shipped UN-watermarked
with the
187         # ctor default paddings; this threads config.json through (the on-by-default
mark).
188         st = cfg.get("stitching") or {}
189         VideoCompiler(
190             local_work_dir,

```

```

191         begin_padding=float(st.get("begin_padding", 0.5)),
192         end_padding=float(st.get("end_padding", 0.5)),
193         watermark=st.get("watermark"),
194     ).compile_highlights(stitch_src, pairs, reel_name)
195     _atomic_publish(local_reel, dest_reel)
196     try:
197         os.remove(local_reel)
198     except OSError:
199         pass
200     logger.info("PUBLISHED reel: %s (%d rallies)", dest_reel, len(pairs))
201     summary["done"].append(name)
202 except Exception:
203     logger.error("ERROR on %s:\n%s", name, traceback.format_exc())
204     summary["failed"].append(name)
205 finally:
206     logging.getLogger().removeHandler(run_fh)
207     run_fh.close()
208     # publish this video's run.log (additive; never deletes anything in out_dir)
209     try:
210         _atomic_publish(run_log_local, os.path.join(out_dir, f"{stem} -
run.log"))
211     except OSError:
212         pass
213     if staged_src and os.path.exists(staged_src):
214         try:
215             os.remove(staged_src)
216         except OSError:
217             pass
218
219     logging.getLogger().removeHandler(batch_fh)
220     batch_fh.close()
221     try:
222         _atomic_publish(batch_log_local, os.path.join(out_dir, "_batch_run.log"))
223 except OSError:
224     pass
225     logger.info("DONE. %s", {k: len(v) for k, v in summary.items()})
226     return summary
227
228
229 def _resolve_videos(args) -> List[str]:
230     if args.video:
231         return [v if os.path.isabs(v) else os.path.join(args.videos_dir or ".", v) for v
in args.video]
232     if args.videos_dir:
233         return sorted(
234             os.path.join(args.videos_dir, f) for f in os.listdir(args.videos_dir)
235             if f.endswith(_VIDEO_EXTS) and os.path.isfile(os.path.join(args.videos_dir,
f))
236         )
237     raise SystemExit("provide --videos-dir or one/more --video")
238
239
240 def _resolve_detect_from(args, video_paths: List[str]) -> Optional[List[str]]:
241     """Resolve --detect-from into a list parallel to video_paths (or None if unused).
242
243     Requires explicit --video (not --videos-dir) and a 1:1 count match ? each detection
244     proxy maps to exactly one --video by position."""
245     if not getattr(args, "detect_from", None):
246         return None
247     if not args.video:
248         raise SystemExit("--detect-from requires explicit --video (not --videos-dir)")
249     if len(args.detect_from) != len(args.video):
250         raise SystemExit(f"--detect-from count ({len(args.detect_from)}) must match "
251             f"--video count ({len(args.video)}); pair them 1:1 in order")
252     return [v if os.path.isabs(v) else os.path.join(args.videos_dir or ".", v) for v in

```

```

args.detect_from]
253
254
255     def main() -> None:
256         ap = argparse.ArgumentParser(description="Batch highlight-reel runner (clean, non-
destructive).")
257         ap.add_argument("--videos-dir", help="folder of source videos (all videos in it,
unless --video given)")
258         ap.add_argument("--video", action="append", help="explicit source video
(repeatable)")
259         ap.add_argument("--out", required=True, help="destination folder for reels +
run.logs")
260         ap.add_argument("--segmenter", default="gemini", help="segmenter registry key
(default: gemini)")
261         ap.add_argument("--local-work", default=os.path.join("output", "highlights_work"),
262             help="local staging dir (reels compiled here, then published)")
263         ap.add_argument("--db", default=os.path.join("output", "sports_indexer.db"))
264         ap.add_argument("--force", action="store_true", help="regenerate even if a reel
already exists")
265         ap.add_argument("--skip-ai-handoff", action="store_true",
266             help="FULLY-LOCAL: trajectory-hybrid segmenters emit candidate
windows as "
267                 "rallies with NO Gemini handoff (no API key / upload). Ignored
by --segmenter gemini.")
268         ap.add_argument("--wasb-stream", action="store_true",
269             help="WASB decode-on-the-fly fast path (#178; output-preserving),
instead of "
270                 "dumping ~46k PNGs. Only affects wasb_hybrid.")
271         ap.add_argument("--detect-from", action="append",
272             help="detection source ? e.g. a 1080p proxy. The detector analyzes
THIS "
273                 "(cheaper to decode) while the reel is stitched from the
matching --video "
274                 "at full resolution (e.g. the 4K original). Repeat to pair 1:1
with --video "
275                 "in the same order. Timestamps transfer 1:1 (pure temporal
downscale).")
276         args = ap.parse_args()
277         video_paths = _resolve_videos(args)
278         detect_from_paths = _resolve_detect_from(args, video_paths)
279         generate_highlights_batch(video_paths, args.out, segmenter=args.segmenter,
280             local_work_dir=args.local_work, db_path=args.db,
force=args.force,
281             skip_ai_handoff=args.skip_ai_handoff,
wasb_stream=args.wasb_stream,
282             detect_from_paths=detect_from_paths)
283
284
285     if __name__ == "__main__":
286         main()
287

```

15. assistant (2026-06-18T01:33:40.715Z)

Let me look at a potential bug in `\_resolve\_detect\_from`. When `--detect-from` is used with `--videos-dir` set AND `--video` set, the `detect\_from` paths use `args.videos\_dir` as base. But in `\_resolve\_videos`, when `args.video` is set, video paths use `args.videos\_dir` or `.`. That's consistent.

Let me check `query\_play\_segments` in the database for the `duration\_min` filter and the compile `segment\_ids` handling, plus pagination logic.

16. user (2026-06-18T01:33:42.718Z)

```

389:     def query_play_segments(self, video_id: str, filters: Dict[str, Any]) -> List[Dict[str,

```

```

Any]]:
392:         - duration_min: float
400:         if filters.get("duration_min"):
402:             params.append(filters["duration_min"])
448:     def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
531:     def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
591:     def claim_due_job(self) -> Optional[Dict[str, Any]]:
602:         "ORDER BY COALESCE(next_poll_at, created_at) ASC LIMIT 1").fetchone()

```

17. user (2026-06-18T01:33:45.690Z)

```

389     def query_play_segments(self, video_id: str, filters: Dict[str, Any]) ->
List[Dict[str, Any]]:
390         """
391         Dynamically queries play segments based on filters:
392         - duration_min: float
393         - server: string
394         - receiver: string
395         - event_type: string (e.g. smash, foul)
396         """
397         query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
398         params = [video_id]
399
400         if filters.get("duration_min"):
401             query += " AND r.duration >= ?"
402             params.append(filters["duration_min"])
403
404         if filters.get("server"):
405             query += " AND r.server = ?"
406             params.append(filters["server"])
407
408         if filters.get("receiver"):
409             query += " AND r.receiver = ?"
410             params.append(filters["receiver"])
411
412         if filters.get("event_type"):
413             # Check if there is an event of a specific type inside this segment
414             query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
415             params.append(filters["event_type"])
416
417         segments_list = []
418         with self._connect() as conn:
419             cursor = conn.cursor()
420             rows = cursor.execute(query, params).fetchall()
421
422             for row in rows:
423                 r_dict = dict(row)
424                 r_id = r_dict["id"]
425                 r_dict["metadata"] = json.loads(r_dict["metadata"]) or "{}"
426
427                 # Fetch associated events
428                 event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
(r_id,)).fetchall()
429                 r_dict["events"] = []
430                 for er in event_rows:
431                     e_dict = dict(er)
432                     e_dict["details"] = json.loads(e_dict["details"]) or "{}"
433                     r_dict["events"].append(e_dict)
434
435                 segments_list.append(r_dict)
436
437         return segments_list
438
439     def clear_video_data(self, video_id: str):

```

```

440         with self._connect() as conn:
441             conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
442             conn.commit()
443
444         # --- Async job store (DEFERRED_HARDENING_PLAN #16) -----
445         # Writers follow the repo convention (swallow + logger.error, return bool); the
446         # worker logs loudly on a failed transition so a job can't silently wedge.
447
448         def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,

```

18. assistant (2026-06-18T01:33:50.663Z)

I notice a subtle bug in `query\_play\_segments`: `if filters.get("duration\_min"):` ? this is a truthiness check, not a `is not None` check. If a user passes `duration\_min=0` (or `0.0`), the filter is silently dropped. That's a minor edge case but arguably intended (0 means no filter). More importantly, the `QueryRequest` allows `duration\_min: Optional[float]`, so a `0.0` would be a no-op filter. This is low-severity and likely intended behavior. Let me keep looking for sharper issues.

Let me check the `upload\_part` more carefully and the other CLI tools. Let me look at the rally_slicer and the remaining argparse scripts.

19. user (2026-06-18T01:33:51.496Z)

```

1         """Slice individual rally clips out of a (long) annotated video.
2
3         Use cases:
4         - **Offline verification** ? pull rally N out of a 30-min match to eyeball its
5           boundaries against the labels.
6         - **Per-video calibration/training** ? grab a handful of labeled rallies as the
7           supervision the segmenter is tuned/adapted on for that specific video.
8
9         Given a labels CSV (the golden `rally_number,start_time,end_time[,start_mmss,
10          end_mmss]` shape) and a set of rally numbers, it writes one padded clip per rally:
11          <out_dir>/<video_stem>__rally_<n>.mp4
12
13         Padding matches the highlight compiler (begin 0.5 s / end 1.0 s) so a sliced clip
14         shows the same lead-in/out the model is trained and evaluated against. The window
15         math (`compute_clip_windows`) is pure and unit-tested; ffmpeg is only touched in
16         `slice_rallies`.
17         """
18         from __future__ import annotations
19
20         import argparse
21         import csv
22         import os
23         import subprocess
24         from dataclasses import dataclass
25         from typing import Dict, List, Optional, Tuple
26
27         from backend.eval.annotations import parse_timestamp
28
29         # Match VideoCompiler defaults so sliced clips frame a rally the same way.
30         BEGIN_PADDING = 0.5
31         END_PADDING = 1.0
32
33
34         @dataclass
35         class ClipWindow:
36             rally: str          # kept as a string to allow inserted ids like "7.5"
37             start: float        # padded, clamped
38             end: float          # padded, clamped
39
40
41         def _read_time(row: Dict[str, str], decimal_key: str, mmss_key: str) -> Optional[float]:

```

```

42     """Prefer the decimal-seconds column; fall back to mm:ss. None if unusable."""
43     for key in (decimal_key, mmss_key):
44         val = (row.get(key) or "").strip()
45         if val:
46             try:
47                 return parse_timestamp(val)
48             except ValueError:
49                 continue
50     return None
51
52
53 def load_rally_labels(csv_path: str) -> Dict[str, Tuple[float, float]]:
54     """Map rally_number -> (start, end) seconds, skipping rows with no usable times.
55
56     Accepts the golden schema (start_time/end_time decimal, optional start_mmss/
57     end_mmss). Rally numbers are kept as strings so inserted ids ("7.5") survive.
58     """
59     labels: Dict[str, Tuple[float, float]] = {}
60     with open(csv_path, newline="", encoding="utf-8") as f:
61         reader = csv.DictReader(f)
62         reader.fieldnames = [(n or "").strip().lower() for n in (reader.fieldnames or
63 [])]
64         for row in reader:
65             row = {(k or "").strip().lower(): v for k, v in row.items()}
66             rally = (row.get("rally_number") or "").strip()
67             if not rally:
68                 continue
69             start = _read_time(row, "start_time", "start_mmss")
70             end = _read_time(row, "end_time", "end_mmss")
71             if start is None or end is None or end <= start:
72                 continue # missing/degenerate (e.g. a not-yet-filled MISSED row)
73             labels[rally] = (start, end)
74     return labels
75
76 def compute_clip_windows(labels: Dict[str, Tuple[float, float]],
77 rally_numbers: List[str],
78 begin_pad: float = BEGIN_PADDING,
79 end_pad: float = END_PADDING,
80 video_duration: Optional[float] = None) -> List[ClipWindow]:
81     """Apply padding and clamp to the video, for the requested rallies.
82
83     rally_numbers may be ["all"] to take every labeled rally (in time order).
84     Unknown rally ids are skipped (caller can diff to warn).
85     """
86     if len(rally_numbers) == 1 and rally_numbers[0].lower() == "all":
87         wanted = sorted(labels, key=lambda r: labels[r][0])
88     else:
89         wanted = [r for r in rally_numbers if r in labels]
90
91     windows: List[ClipWindow] = []
92     for rally in wanted:
93         start, end = labels[rally]
94         ps = max(0.0, start - begin_pad)
95         pe = end + end_pad
96         if video_duration is not None and video_duration > 0:
97             if ps >= video_duration:
98                 continue # entirely past the end
99             pe = min(pe, video_duration)
100         if pe <= ps:
101             continue
102         windows.append(ClipWindow(rally=rally, start=round(ps, 3), end=round(pe, 3)))
103     return windows
104
105

```

```

106 def _probe_duration(video_path: str) -> Optional[float]:
107     try:
108         out = subprocess.run(
109             ["ffprobe", "-v", "error", "-show_entries", "format=duration",
110              "-of", "default=noprint_wrappers=1:nokey=1", video_path],
111             capture_output=True, text=True, check=True,
112             ).stdout.strip()
113         return float(out)
114     except (subprocess.SubprocessError, OSError, ValueError):
115         return None
116
117
118 def slice_rallies(video_path: str, labels_csv: str, rally_numbers: List[str],
119                  out_dir: str, begin_pad: float = BEGIN_PADDING,
120                  end_pad: float = END_PADDING) -> List[str]:
121     """Write one padded clip per requested rally; returns the clip paths written."""
122     if not os.path.exists(video_path):
123         raise FileNotFoundError(f"video not found: {video_path}")
124     labels = load_rally_labels(labels_csv)
125     if not labels:
126         raise ValueError(f"no usable rally rows in {labels_csv}")
127     os.makedirs(out_dir, exist_ok=True)
128
129     duration = _probe_duration(video_path)
130     windows = compute_clip_windows(labels, rally_numbers, begin_pad, end_pad, duration)
131     if not windows:
132         raise ValueError("no rallies matched the request (check --rallies vs the CSV)")
133
134     stem = os.path.splitext(os.path.basename(video_path))[0]
135     written: List[str] = []
136     for w in windows:
137         safe_rally = w.rally.replace(".", "_")
138         out_path = os.path.join(out_dir, f"{stem}__rally_{safe_rally}.mp4")
139         cmd = [
140             "ffmpeg", "-y",
141             "-ss", str(w.start), "-to", str(w.end), "-i", video_path,
142             "-c:v", "libx264", "-preset", "superfast", "-crf", "22",
143             "-c:a", "aac", "-b:a", "128k",
144             out_path,
145         ]
146         try:
147             subprocess.run(cmd, capture_output=True, check=True)
148         except subprocess.CalledProcessError as e:
149             err = e.stderr.decode(errors="replace").strip()[-300:]
150             print(f"[slicer] rally {w.rally}: ffmpeg failed: {err}")
151             continue
152         if os.path.exists(out_path) and os.path.getsize(out_path) > 0:
153             written.append(out_path)
154             print(f"[slicer] rally {w.rally} -> {out_path}")
155     return written
156
157
158 def main() -> None:
159     p = argparse.ArgumentParser(description="Slice rally clips from an annotated
160 video.")
161     p.add_argument("--video", required=True, help="Source video file")
162     p.add_argument("--labels", required=True, help="Labels CSV (golden rally format)")
163     p.add_argument("--rallies", required=True,
164                     help="Comma-separated rally numbers (e.g. 7,7.5,16) or 'all'")
165     p.add_argument("--out-dir", default="output/rally_clips", help="Output directory")
166     p.add_argument("--begin-pad", type=float, default=BEGIN_PADDING)
167     p.add_argument("--end-pad", type=float, default=END_PADDING)
168     args = p.parse_args()

```

```

169         rally_numbers = [r.strip() for r in args.rallies.split(",") if r.strip()]
170         written = slice_rallies(args.video, args.labels, rally_numbers,
171                                args.out_dir, args.begin_pad, args.end_pad)
172         print(f"\nWrote {len(written)} clip(s) to {args.out_dir}")
173
174
175     if __name__ == "__main__":
176         main()
177

```

20. assistant (2026-06-18T01:33:54.412Z)

Let me look at the remaining argparse CLI scripts and check `create\_job` / `get\_job` for type coercion issues with `max\_frames` and `player\_pool`.

21. user (2026-06-18T01:33:55.588Z)

```

448     def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
449                   player_pool: Optional[List[str]] = None,
450                   max_frames: Optional[int] = None,
451                   policy: Optional[Dict[str, Any]] = None) -> bool:
452         """Insert a new job in state 'queued'. ``policy`` is an optional JSON
ExecutionPolicy
453         (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy."""
454         try:
455             with self._connect() as conn:
456                 conn.cursor().execute(
457                     "INSERT INTO jobs (id, video_path, segmenter, player_pool,
max_frames, status, policy) "
458                     "VALUES (?, ?, ?, ?, ?, 'queued', ?)",
459                     (job_id, video_path, segmenter,
460                      json.dumps(player_pool) if player_pool is not None else None,
461                      max_frames, json.dumps(policy) if policy is not None else None)
462                 )
463                 conn.commit()
464             return True
465         except Exception as e:
466             logger.error(f"Failed to create job {job_id}: {e}")
467             return False
468
469     def mark_job_running(self, job_id: str) -> bool:
470         try:
471             with self._connect() as conn:
472                 conn.cursor().execute(
473                     "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP, "
474                     "progress='starting' WHERE id = ?", (job_id,))
475                 conn.commit()
476             return True
477         except Exception as e:
478             logger.error(f"Failed to mark job {job_id} running: {e}")
479             return False
480
481     def set_job_progress(self, job_id: str, progress: str) -> bool:
482         try:
483             with self._connect() as conn:
484                 conn.cursor().execute(
485                     "UPDATE jobs SET progress=? WHERE id = ?", (progress, job_id))
486                 conn.commit()
487             return True
488         except Exception as e:
489             logger.error(f"Failed to set progress for job {job_id}: {e}")
490             return False
491
492     def set_job_video_id(self, job_id: str, video_id: str) -> bool:
493         try:

```



```

494         with self._connect() as conn:
495             conn.cursor().execute(
496                 "UPDATE jobs SET video_id=? WHERE id = ?", (video_id, job_id))
497             conn.commit()
498         return True
499     except Exception as e:
500         logger.error(f"Failed to set video_id for job {job_id}: {e}")
501         return False
502
503     def mark_job_done(self, job_id: str, result: Dict[str, Any]) -> bool:
504         """Job EXECUTED to completion. The pipeline's own verdict (success/failed
505         validation) is inside `result` ? job status 'done' means 'the worker
506         finished'."""
507         try:
508             with self._connect() as conn:
509                 conn.cursor().execute(
510                     "UPDATE jobs SET status='done', progress='finished', result=?, "
511                     "finished_at=CURRENT_TIMESTAMP WHERE id = ?",
512                     (json.dumps(result), job_id))
513                 conn.commit()
514             return True
515         except Exception as e:
516             logger.error(f"Failed to mark job {job_id} done: {e}")
517             return False
518
519     def mark_job_failed(self, job_id: str, error: str) -> bool:
520         """Worker crashed / raised ? infrastructure failure, not a pipeline verdict."""
521         try:
522             with self._connect() as conn:
523                 conn.cursor().execute(
524                     "UPDATE jobs SET status='failed', error=?, "
525                     "finished_at=CURRENT_TIMESTAMP WHERE id = ?", (error, job_id))
526                 conn.commit()
527             return True
528         except Exception as e:
529             logger.error(f"Failed to mark job {job_id} failed: {e}")
530             return False
531
532     def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
533         """Fetch one job; `result`/`player_pool` JSON deserialized."""
534         with self._connect() as conn:
535             row = conn.cursor().execute("SELECT * FROM jobs WHERE id = ?",
536                                         (job_id,)).fetchone()
537             if not row:
538                 return None
539             d = dict(row)
540             d["result"] = json.loads(d["result"]) if d["result"] else None
541             d["player_pool"] = json.loads(d["player_pool"]) if d["player_pool"] else
542             None
543             d["policy"] = json.loads(d["policy"]) if d.get("policy") else None
544             return d
545
546     # --- Self-scheduling execution policy (EXECUTION_POLICY.md P2)
547     -----
548     def set_job_policy(self, job_id: str, policy: Optional[Dict[str, Any]]) -> bool:
549         """Attach/replace a job's JSON ExecutionPolicy."""
550         try:
551             with self._connect() as conn:
552                 conn.cursor().execute(
553                     "UPDATE jobs SET policy=? WHERE id = ?",
554                     (json.dumps(policy) if policy is not None else None, job_id))
555                 conn.commit()
556             return True
557         except Exception as e:
558             logger.error(f"Failed to set policy for job {job_id}: {e}")

```

```

555         return False
556
557     def set_job_waiting(self, job_id: str, delay_sec: float, progress: Optional[str] =
None) -> bool:
558         """Self-schedule: park a job in 'waiting' until ``delay_sec`` from now,
releasing the
559         worker. ``claim_due_job`` re-picks it when due ? cooperative, no master.
``next_poll_at`` is
560         computed in SQLite's own UTC format so it compares directly to
CURRENT_TIMESTAMP."""
561         try:
562             with self._connect() as conn:
563                 conn.cursor().execute(
564                     "UPDATE jobs SET status='waiting', "
565                     "next_poll_at=datetime('now', ?), "
566                     "progress=COALESCE(?, progress) WHERE id = ?",
567                     (f"{max(0, int(delay_sec))} seconds", progress, job_id))
568                 conn.commit()
569             return True
570         except Exception as e:
571             logger.error(f"Failed to set job {job_id} waiting: {e}")
572             return False
573
574     def seconds_until_next_due(self) -> Optional[float]:
575         """Seconds until the soonest parked ('waiting') job becomes due, so the worker
can
576         schedule a wake-up (EXECUTION_POLICY.md P3 self-scheduling). Returns 0.0 if one
is
577         already due, or None if nothing is waiting. A NULL ``next_poll_at`` ? due now
(0.0)."""
578         try:
579             with self._connect() as conn:
580                 row = conn.cursor().execute(
581                     "SELECT MIN(CASE WHEN next_poll_at IS NULL THEN 0 ELSE "
582                     "    (julianday(next_poll_at) - julianday('now')) * 86400.0 END) AS
secs "
583                     "FROM jobs WHERE status='waiting'").fetchone()
584                 if row is None or row["secs"] is None:
585                     return None
586                 return max(0.0, float(row["secs"]))
587         except Exception as e:
588             logger.error(f"Failed to compute next-due delay: {e}")
589             return None
590
591     def claim_due_job(self) -> Optional[Dict[str, Any]]:
592         """Atomically claim ONE runnable job ? 'queued', or 'waiting' whose
``next_poll_at`` is due ?
593         flipping it to 'running'. Compare-and-set on (id, status) so concurrent workers
never
594         double-claim (the DB row-claim is the only coordination needed; no master).
Earliest-due
595         first. Returns the claimed job dict, or None if nothing is runnable."""
596         try:
597             with self._connect() as conn:
598                 cur = conn.cursor()
599                 row = cur.execute(
600                     "SELECT id, status FROM jobs WHERE status='queued' "
601                     "OR (status='waiting' AND (next_poll_at IS NULL OR next_poll_at <=
CURRENT_TIMESTAMP)) "
602                     "ORDER BY COALESCE(next_poll_at, created_at) ASC LIMIT
1").fetchone()
603                 if not row:
604                     return None
605                 cur.execute(
606                     "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP "

```

```

607         "WHERE id=? AND status=?", (row["id"], row["status"]))
608         conn.commit()
609         if cur.rowcount != 1:
610             return None                    # lost the race to a concurrent
worker
611         return self.get_job(row["id"])
612     except Exception as e:
613         logger.error(f"Failed to claim a due job: {e}")
614         return None
615
616     def fail_stale_jobs(self, reason: str = "interrupted by server restart") -> int:
617         """Crash recovery: any job still queued/running from a previous process is dead

```

22. assistant (2026-06-18T01:34:00.085Z)

Now let me focus on the `process\_video` endpoint's validation. There's a subtle issue: `max\_frames` is `Optional[int]` in the Pydantic model, but there's no validation that it's positive. Also, `validate\_input\_path` is called before `os.path.exists` ? but the order matters for traversal. Let me check whether the `ProcessRequest` validation handles non-existent dirs.

Let me now look at the remaining CLI scripts with argparse to be thorough, and examine the `wasb\_infer.py` and `run\_phase3\_eval.py`.

23. user (2026-06-18T01:34:01.157Z)

```

1     """Phase 3 driver: batch process + evaluate a collector export against ground truth.
2
3     Walks the `manifest.json` produced by sports-data-collector's `curate export`,
4     processes each video through a segmenter (so detections land in the indexer DB),
5     scores each against its ground-truth rally CSV, and prints per-video plus
6     corpus-aggregate precision/recall/F1, mean IoU, and boundary error.
7
8     Examples
9     -----
10    Free CPU baseline on a single video (validate the loop):
11        python run_phase3_eval.py --manifest ../sports-data-
collector/data/eval_export/manifest.json \
12        --segmenter motion --only shuttlest_1
13
14    Full corpus with the two-stage detector (needs GEMINI_API_KEY; costs API calls):
15        python run_phase3_eval.py --manifest ../manifest.json --segmenter yolo_hybrid \
16        --json output/phase3_report.json
17
18    Re-score only (videos already processed), no re-segmentation:
19        python run_phase3_eval.py --manifest ../manifest.json --score-only
20    """
21
22    import os
23    import sys
24    import json
25    import time
26    import argparse
27    import logging
28
29    from dotenv import load_dotenv
30    load_dotenv() # make GEMINI_API_KEY (etc.) available to the AI-backed segmenters
31
32    from backend.infrastructure.database import Database
33    from backend.config import load_config
34    from backend.utils.hashing import compute_video_id as get_file_md5 # canonical video_id
35    # Importing the package registers all segmenters (motion/gemini/yolo_hybrid/...).
36    from backend.pipeline.segmenters import segmenter_registry
37    from backend.eval.annotations import load_annotations
38    from backend.eval.harness import evaluate, format_report_text, report_to_dict
39    from backend.eval import batch

```

```

40
41     logger = logging.getLogger(__name__)
42
43
44     def build_parser() -> argparse.ArgumentParser:
45         p = argparse.ArgumentParser(
46             description="Batch process + evaluate a collector export manifest (Phase 3).",
47             formatter_class=argparse.RawDescriptionHelpFormatter,
48         )
49         p.add_argument("--manifest", required=True, help="Path to the collector's
manifest.json.")
50         p.add_argument("--videos-root", default=None,
51             help="Root for resolving manifest relative local_paths "
52             "(default: the collector repo root, two levels above the
manifest).")
53         p.add_argument("--segmenter", default="motion",
54             help="Segmenter to run (default: motion ? free/CPU). "
55             "yolo_hybrid/gemini need GEMINI_API_KEY and cost API calls.")
56         p.add_argument("--stage", choices=("candidates", "final", "both", "auto"),
57             default="auto",
58             help="Which prediction stage(s) to score (default: auto by
segmenter).")
59         p.add_argument("--detector", default=None, help="Restrict candidate scoring to this
detector.")
60         p.add_argument("--iou", type=float, default=0.5, help="IoU match threshold (default:
0.5).")
61         p.add_argument("--only", action="append", default=None,
62             help="Only this video_id (repeatable).")
63         p.add_argument("--limit", type=int, default=None, help="Process at most N videos.")
64         p.add_argument("--max-frames", type=int, default=None,
65             help="Cap segmenter frames (fast validation).")
66         p.add_argument("--score-only", action="store_true",
67             help="Skip segmentation; only score detections already in the DB.")
68         p.add_argument("--reprocess", action="store_true",
69             help="Re-run segmentation even if the video already has segments.")
70         p.add_argument("--db", default=None, help="Indexer DB path (default:
output/<db_name>).")
71         p.add_argument("--json", dest="json_out", default=None, help="Write a JSON report
here.")
72         p.add_argument("--show-failures", action="store_true", help="List missed/spurious
rallies per video.")
73         return p
74
75     def main(argv=None) -> int:
76         logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
77         args = build_parser().parse_args(argv)
78
79         manifest_path = os.path.abspath(args.manifest)
80         if not os.path.exists(manifest_path):
81             print(f"ERROR: manifest not found: {manifest_path}", file=sys.stderr)
82             return 2
83         manifest_dir = os.path.dirname(manifest_path)
84         videos_root = args.videos_root or os.path.dirname(os.path.dirname(manifest_dir))
85
86         with open(manifest_path) as f:
87             manifest = json.load(f)
88             entries = manifest.get("eval_sets", [])
89
90         cfg = load_config() # typed AppConfig (same object the live pipeline feeds
segmenters)
91         db_path = args.db or os.path.join(cfg.output.output_dir, cfg.output.db_name)
92         os.makedirs(os.path.dirname(db_path) or ".", exist_ok=True)
93         db = Database(db_path)
94

```

```

95     stages = batch.default_stages_for(args.segmenter, args.stage)
96
97     # Resolve the segmenter class up front (unless purely scoring).
98     segmenter = None
99     if not args.score_only:
100         seg_cls = segmenter_registry.get(args.segmenter)
101         if not seg_cls:
102             print(f"ERROR: segmenter '{args.segmenter}' not found. "
103                   f"Available: {list(segmenter_registry.list_available().keys())}",
104                   file=sys.stderr)
105             return 2
106         segmenter = seg_cls(db, cfg)
107
108     # Filter entries.
109     targets = []
110     for e in entries:
111         if args.only and e["video_id"] not in args.only:
112             continue
113         targets.append(e)
114     if args.limit:
115         targets = targets[: args.limit]
116
117     per_video = []
118     skipped = []
119     print(f"Phase 3: {len(targets)} target(s) | segmenter={args.segmenter} |
120 stages={stages} | db={db_path}\n")
121
122     for e in targets:
123         vid = e["video_id"]
124         video_path = batch.resolve_video_path(e.get("local_path"), videos_root)
125         csv_path = os.path.join(manifest_dir, e["csv"])
126
127         if not video_path or not os.path.exists(video_path):
128             skipped.append((vid, "no video file"))
129             print(f"[skip] {vid}: no video file ({e.get('local_path')})")
130             continue
131         if not os.path.exists(csv_path):
132             skipped.append((vid, "no GT csv"))
133             print(f"[skip] {vid}: GT csv missing ({csv_path})")
134             continue
135
136         video_id = get_file_md5(video_path)
137
138         # Segment unless score-only or already present (and not --reprocess).
139         if not args.score_only:
140             already = db.query_play_segments(video_id, {}) or
141             db.query_candidate_segments(video_id)
142             if already and not args.reprocess:
143                 print(f"[have] {vid}: detections already in DB (use --reprocess to
144 redo)")
145             else:
146                 print(f"[proc] {vid}: segmenting with {args.segmenter}...")
147                 db.add_video(video_id, video_path, "processed", [])
148                 t0 = time.time()
149                 segs = segmenter.process_video(video_id, video_path,
150 max_frames=args.max_frames)
151                 print(f"          -> {len(segs)} segments in {time.time() - t0:.0f}s")
152
153         gt = load_annotations(csv_path)
154         report = evaluate(db, video_id, gt, stages, iou_threshold=args.iou,
155 detector=args.detector)
156         print(format_report_text(report, show_failures=args.show_failures))
157         print()
158         d = report_to_dict(report)
159         d["collector_video_id"] = vid

```

```

154         per_video.append(d)
155
156     agg = batch.aggregate(per_video, stages)
157     print(batch.format_aggregate(agg))
158     if skipped:
159         print(f"\nSkipped {len(skipped)}: " + ", ".join(f"{v} ({why})" for v, why in
skipped))
160
161     if args.json_out:
162         with open(args.json_out, "w") as f:
163             json.dump({"segmenter": args.segmenter, "iou": args.iou, "stages": stages,
164                       "aggregate": agg, "per_video": per_video,
165                       "skipped": skipped}, f, indent=2)
166             print(f"\nJSON report written to {args.json_out}")
167
168     return 0
169
170
171 if __name__ == "__main__":
172     raise SystemExit(main())
173

```

24. assistant (2026-06-18T01:34:07.652Z)

Line 145 has a potential bug: `segs = segmenter.process\_video(...)` then `len(segs)` ?
`process\_video` can return a `Failure` object (as seen in main.py), and `len(Failure)` would crash. But this is the eval driver path; let me check if it's in scope. Actually it's in `scripts/` not excluded, but let me focus ? `process\_video` returning Failure and `len()` on it would be a TypeError. But this depends on whether the segmenter returns Failure. Let me note this but verify it's a real concern. The main.py code uses `isinstance(result, Failure)` guard, while this script does `len(segs)` directly. If the segmenter fails, this crashes with `TypeError: object of type 'Failure' has no len()`. That's a real defect but lower severity (eval tooling, crashes loudly). Let me check the Failure class.

25. user (2026-06-18T01:34:08.491Z)

```

1      """Lightweight Result type for standardizing success vs. failure across the pipeline.
2
3      This is the start of the Error/Result Contract standardization (low-regret item #3).
4
5      The goal is to make it impossible to confuse "no data / no rallies" with "something
actually failed"
6      (e.g. missing API key, WSL healthcheck failure, provider error, etc.).
7
8      We use simple dataclasses so callers can use isinstance checks or structural pattern
matching.
9      This is intentionally small and incremental ? we start by applying it to the segmenter
10     and provider layers (the biggest sources of silent [] returns today).
11
12     The types are generic where it makes sense. Callers can still access .value on Success
13     or .message on Failure.
14
15     This module has no dependencies on the rest of the app.
16     """
17
18     from __future__ import annotations
19
20     from dataclasses import dataclass, field
21     from typing import Generic, TypeVar, List, Optional, Any, Union
22
23     T = TypeVar("T")
24
25
26     @dataclass
27     class Success(Generic[T]):

```

```

28         """Represents a successful outcome with an optional list of non-fatal warnings."""
29
30     value: T
31     warnings: List[str] = field(default_factory=list)
32
33     def is_success(self) -> bool:
34         return True
35
36     def unwrap(self) -> T:
37         """Return the value. Only safe to call if you already know it's a Success."""
38         return self.value
39
40
41     @dataclass
42     class Failure:
43         """Represents a failure with a human-readable message and optional details."""
44
45         message: str
46         details: Optional[dict[str, Any]] = None
47         is_recoverable: bool = False # e.g. missing API key (recoverable) vs internal crash
48
49         def is_success(self) -> bool:
50             return False
51
52         def unwrap(self) -> Any:
53             """Raise to make misuse obvious in code that assumes success."""
54             raise RuntimeError(f"Attempted to unwrap a Failure: {self.message}")
55
56
57     # Convenient alias for use in type hints: Result[list[dict], Failure]
58     # Result[ValueType] means Success[ValueType] | Failure
59     # (the Failure "error" type is fixed for this contract)
60     Result = Union[Success[T], Failure]
61

```

26. assistant (2026-06-18T01:34:13.046Z)

Now let me confirm: does `process\_video` return a `Failure` or a `Success`/list? In main.py, the code does `segments = result.value if hasattr(result, "value") else result`. So `process\_video` returns either a `Failure`, a `Success` (with `.value`), or a bare list. In `run\_phase3\_eval.py` line 145-146, it does `len(segs)` directly without unwrapping `.value` or checking `Failure`.

This means:

- If `process\_video` returns `Success(value=[...])`, `len(Success)` ? TypeError (Success has no `\_\_len\_\_`)
- If it returns `Failure`, `len(Failure)` ? TypeError

But wait ? let me check what `process\_video` actually returns. Let me look at the base segmenter and motion segmenter.

27. user (2026-06-18T01:34:14.633Z)

<persisted-output>

Output too large (20.4KB). Full output saved to: C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\tool-results\toolu_01WvKRnmm3RXik6uhhsprLtLk.txt

Preview (first 2KB):

```

.claude\worktrees\hunt-base\backend\pipeline\segmenters\base.py:24:     def process_video(self,
video_id: str, video_path: str, player_pool: List[str] = None, max_frames: int = None) ->
"Result[List[Dict[str, Any]]]":
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\pipeline\segmenters\base.py-25-         """
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\pipeline\segmenters\base.py-26-         Processes a video to detect play segments,

```

```

populates SQLite database,
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\pipeline\segmenters\base.py-27-         and returns a Result.
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\pipeline\segmenters\base.py-28-
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\pipeline\segmenters\base.py-29-         Use isinstance(result, Failure) (or
result.is_success()) to distinguish
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\pipeline\segmenters\base.py-30-         "no segments found" from "something went
wrong" (missing key, healthcheck
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\pipeline\segmenters\base.py-31-         failure, etc.). This is the start of the
standardized error/result contract.
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\pipeline\segmenters\base.py-32-         ""
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\pipeline\segmenters\base.py-33-         pass
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\pipeline\segmenters\base.py-34-
.claude\worktrees\hunt-base\backend\pipeline\segmenters\base.py-35-class SegmenterRegistry:
.claude\worktrees\hunt-base\backend\pipeline\segmenters\base.py-36-    def __init__(self):
...
</persisted-output>

```

28. assistant (2026-06-18T01:34:19.031Z)

So `process\_video` returns `Result[List[Dict]] = Success[List] | Failure`. The eval driver `run\_phase3\_eval.py` line 145-146 calls `len(segs)` directly on the return value without unwrapping. This is a real bug ? but let me check whether the motion segmenter actually returns a bare list vs a Success. Let me check the actual return statements.

29. user (2026-06-18T01:34:20.791Z)

```

backend/pipeline/segmenters/gemini.py:46:         return []
backend/pipeline/segmenters/gemini.py:62:         return []
backend/pipeline/segmenters/gemini.py:68:         return []
backend/pipeline/segmenters/gemini.py:303:        return []
backend/pipeline/segmenters/gemini.py:469:        return segments_data
backend/pipeline/segmenters/motion.py:31:        return Failure(message=f"Could not open
video: {video_path}", is_recoverable=False)
backend/pipeline/segmenters/motion.py:224:        return Success(value=segments_data)
backend/pipeline/segmenters/trajectory_hybrid.py:119:        return []
backend/pipeline/segmenters/trajectory_hybrid.py:144:        return []
backend/pipeline/segmenters/trajectory_hybrid.py:149:        return []
backend/pipeline/segmenters/trajectory_hybrid.py:154:        return []
backend/pipeline/segmenters/trajectory_hybrid.py:189:        return []
backend/pipeline/segmenters/trajectory_hybrid.py:200:        return []
backend/pipeline/segmenters/trajectory_hybrid.py:247:        return []
backend/pipeline/segmenters/trajectory_hybrid.py:283:        return []
backend/pipeline/segmenters/trajectory_hybrid.py:337:        return segments_data
backend/pipeline/segmenters/yolo_hybrid.py:66:        return []
backend/pipeline/segmenters/yolo_hybrid.py:82:        return []
backend/pipeline/segmenters/yolo_hybrid.py:88:        return []
backend/pipeline/segmenters/yolo_hybrid.py:95:        return []
backend/pipeline/segmenters/yolo_hybrid.py:288:        return []
backend/pipeline/segmenters/yolo_hybrid.py:366:        return segments_data

```

30. assistant (2026-06-18T01:34:30.187Z)

Confirmed. The `motion` segmenter returns `Success(value=...)` (line 224) or `Failure` (line 31). So in `run\_phase3\_eval.py`, when using `--segmenter motion`, `segs = segmenter.process\_video(...)` returns a `Success` object, and `len(segs)` on line 146 raises `TypeError: object of type 'Success' has no len()`.

This is a real, reachable bug. The default segmenter in this script is `motion` (line 53), which returns `Success`. So `python run\_phase3\_eval.py --manifest ...` (the documented "Free CPU baseline" example on line 11-12) crashes on `len(segs)` for any video that actually gets processed (not score-only, not already-have). However ? let me verify it's reachable: it needs `--score-only` NOT set, and the video not already in DB. The default flow processes. This is a genuine defect.

Let me verify there's no `\_\_len\_\_` on Success ? confirmed, it's a plain dataclass with `value` and `warnings`. So `len(Success(...))` raises TypeError.

Note: `gemini`, `yolo\_hybrid`, `trajectory\_hybrid` return bare lists, so `len()` works for those ? but `motion` (the default and the documented example) returns `Success`. This is a real bug for the default path.

Let me confirm there's no test masking this and check `git log` to see if this is recent. Let me also double check this isn't excluded ? `scripts/run\_phase3\_eval.py` is NOT in the exclusion list. Good. Let me now look at the other files for the CLI/argparse class quickly: `wasb\_infer.py`, `calibrate\_\*`, `cleanup\_caches.py`.

31. user (2026-06-18T01:34:31.981Z)

```
scripts/run_phase3_eval.py:145:             segs = segmenter.process_video(video_id,
video_path, max_frames=args.max_frames)
scripts/run_phase3_eval.py:146:             print(f"             -> {len(segs)} segments in
{time.time() - t0:.0f}s")
```

32. assistant (2026-06-18T01:34:35.947Z)

Confirmed ? no unwrapping. This is a real bug. Now let me check the other CLI files for similar patterns and the FastAPI compile endpoint's `segment\_ids` handling.

Let me look at `wasb\_infer.py`, `calibrate\_local.py`, `calibrate\_wasb.py`, and `cleanup\_caches.py` for argparse issues.

33. user (2026-06-18T01:34:36.089Z)

```
1      """Cache-hygiene tool for the GPU highlight pipeline.
2
3      Scans the regenerable intermediates the pipeline leaves behind ? in the WSL stage
4      dir (`~/clips`) and the Windows `output/` tree ? classifies each entry as
5      **purge** (regenerable: frame caches, staged video copies, handoff clips, temp dirs)
6      or **keep** (durable: trajectory CSVs, the SQLite DB, final reels) and reports the
7      reclaimable space.
8
9      DRY-RUN BY DEFAULT ? it only *shows* what it would delete. Pass `--apply` to delete.
10
11         python -m tools.cleanup_caches             # report only (safe)
12         python -m tools.cleanup_caches --apply      # delete purge candidates
13         python -m tools.cleanup_caches --keep-video 89605e80ed01 # protect one video's
cache
14         python -m tools.cleanup_caches --older-than 7      # only purge entries >7 days old
15         python -m tools.cleanup_caches --include-wasbcache # also drop detector resume
caches
16
17         Pairs with the pipeline's own self-cleaning (indexing.{wasb,tracknet}.keep_frames=false,
18         which deletes frames after a SUCCESSFUL run); this tool reclaims what failed/old runs
19         and pre-self-cleaning videos left behind. Stdlib only; safe to run anywhere (the WSL
20         scan is skipped automatically when `wsl` is unavailable).
21         """
22         from __future__ import annotations
23
24         import argparse
25         import os
26         import shutil
```

```

27     import subprocess
28     import sys
29     from dataclasses import dataclass
30     from typing import List, Optional, Tuple
31
32     # ---- classification (pure; unit-tested) ----- #
33
34     # (category, regenerable, default_purge). default_purge=True ? deleted unless a flag
35     # protects it. wasbcache is regenerable but default_purge=False (it enables resume and
36     # is tiny) ? opt in with --include-wasbcache.
37     _KEEP = ("keep", False, False)
38
39
40     def classify(name: str, is_dir: bool, location: str) -> Tuple[str, bool, bool]:
41         """Classify a cache entry by name. Returns (category, regenerable,
42         default_purge)."""
43         low = name.lower()
44         if is_dir:
45             if low.endswith("_frames"):
46                 return ("frame cache", True, True)
47             if low.endswith("_wasbcache"):
48                 return ("detector resume cache", True, False) # opt-in via --include-
49         wasbcache
50             if low.startswith("chunks_") and low.endswith("_handoff"):
51                 return ("ai handoff clips", True, True)
52             if low.startswith("tmp"):
53                 return ("temp dir", True, True)
54             return ("other dir", False, False)
55         # files
56         if low.endswith(("csv",)):
57             return ("trajectory csv", False, False) # durable analysis output
58         if low.endswith(("db", ".sqlite", ".sqlite3")):
59             return ("database", False, False) # cached rallies ? never purge
60         if low.endswith(("log", ".sh", ".txt", ".json", ".md")):
61             return ("log/script", False, False)
62         if low.endswith(("mp4", ".avi", ".mov", ".mkv")):
63             # In the WSL stage dir, every video is a regenerable staged copy.
64             if location == "wsl":
65                 return ("staged video", True, True)
66             # In output/, a 1080p/proxy is regenerable; anything else is a produced reel.
67             if "_1080p" in low or "_proxy" in low or low.endswith("_proxy.mp4"):
68                 return ("proxy video", True, True)
69             return ("highlight reel", False, False) # durable output ? keep
70         return ("other", False, False)
71
72     @dataclass
73     class Entry:
74         name: str
75         path: str # display path (WSL ~/clips/<name> or windows abs)
76         size: int # bytes
77         mtime: float # epoch seconds (0 if unknown)
78         is_dir: bool
79         location: str # "wsl" | "windows"
80
81     @property
82     def category(self) -> Tuple[str, bool, bool]:
83         return classify(self.name, self.is_dir, self.location)
84
85     def plan_deletions(entries: List[Entry], *, keep_video: Optional[str], older_than_days:
86     float,
87     include_wasbcache: bool, now: float) -> Tuple[List[Entry],
88     List[Entry]]:
89         """Split entries into (to_delete, kept) per the flags. Pure ? no I/O."""

```

```

88     to_delete: List[Entry] = []
89     kept: List[Entry] = []
90     cutoff = older_than_days * 86400.0
91     for e in entries:
92         cat, _regen, default_purge = e.category
93         purge = default_purge or (include_wasbcache and cat == "detector resume cache")
94         if purge and keep_video and keep_video in e.name:
95             purge = False
96         if purge and older_than_days > 0 and e.mtime > 0 and (now - e.mtime) < cutoff:
97             purge = False
98         (to_delete if purge else kept).append(e)
99     return to_delete, kept
100
101
102 # ---- scanning (I/O) ----- #
103
104 def human(n: int) -> str:
105     f = float(n)
106     for unit in ("B", "K", "M", "G", "T"):
107         if f < 1024 or unit == "T":
108             return f"{f:.1f}{unit}" if unit != "B" else f"{int(f)}B"
109     f /= 1024
110     return f"{f:.1f}T"
111
112
113 def _wsl_lines(distro: str, cmd: str) -> List[str]:
114     """Run a bash command in WSL and return stdout lines ([] on any failure)."""
115     try:
116         res = subprocess.run(["wsl", "-d", distro, "bash", "-lc", cmd],
117                               capture_output=True, text=True, timeout=300)
118     except (OSError, subprocess.TimeoutExpired):
119         return []
120     return (res.stdout or "").splitlines()
121
122
123 def scan_wsl(distro: str, stage_dir: str) -> List[Entry]:
124     """List ~/clips entries with recursive size + mtime + type. [] if WSL absent.
125
126     Uses ``du -sb`` (recursive byte sizes, native-fast on ext4) merged with ``find
127     -printf`` (type + mtime) by basename. Deliberately avoids a ``for`` loop ? the
128     user's login shell mangles loop variables, but du/find arg-globbing is reliable.
129     """
130     if not shutil.which("wsl"):
131         return []
132     size_lines = _wsl_lines(distro, f"du -sb {stage_dir}/* {stage_dir}/.[!.*]*
2>/dev/null || true")
133     meta_lines = _wsl_lines(distro, f"find {stage_dir}/ -maxdepth 1 -mindepth 1 "
134                             f"-printf '%y\\t%T@\\t%f\\n' 2>/dev/null || true")
135     meta = {} # basename -> (is_dir, mtime)
136     for line in meta_lines:
137         parts = line.split("\\t")
138         if len(parts) == 3:
139             y, mt, name = parts
140             try:
141                 meta[name] = (y == "d", float(mt))
142             except ValueError:
143                 continue
144     entries: List[Entry] = []
145     for line in size_lines:
146         if "\\t" not in line:
147             continue
148         sz, path = line.split("\\t", 1)
149         name = path.rsplit("/", 1)[-1]
150         try:
151             size = int(sz)

```

```

152         except ValueError:
153             continue
154         is_dir, mtime = meta.get(name, (False, 0.0))
155         entries.append(Entry(name=name, path=f"{stage_dir}/{name}", size=size,
156                             mtime=mtime, is_dir=is_dir, location="wsl"))
157     return entries
158
159
160 def _dir_size(path: str) -> int:
161     total = 0
162     for root, _dirs, files in os.walk(path):
163         for fn in files:
164             try:
165                 total += os.path.getsize(os.path.join(root, fn))
166             except OSError:
167                 pass
168     return total
169
170
171 def scan_windows(output_dir: str) -> List[Entry]:
172     """List output/ entries (top level) plus the output/wasb sub-tree."""
173     entries: List[Entry] = []
174     for base in (output_dir, os.path.join(output_dir, "wasb")):
175         if not os.path.isdir(base):
176             continue
177         for de in os.scandir(base):
178             # 'wasb' is scanned as its own base below ? don't also list it as a dir
179             if base == output_dir and de.name == "wasb":
180                 continue
181             try:
182                 is_dir = de.is_dir()
183                 size = _dir_size(de.path) if is_dir else de.stat().st_size
184                 mtime = de.stat().st_mtime
185             except OSError:
186                 continue
187             entries.append(Entry(name=de.name, path=de.path, size=size,
188                                 mtime=mtime, is_dir=is_dir, location="windows"))
189     return entries
190
191
192 def _delete(e: Entry, distro: str) -> bool:
193     try:
194         if e.location == "wsl":
195             subprocess.run(["wsl", "-d", distro, "bash", "-lc", f'rm -rf {_q(e.path)}'],
196                             capture_output=True, text=True, timeout=600, check=True)
197         elif e.is_dir:
198             shutil.rmtree(e.path, ignore_errors=True)
199         else:
200             os.remove(e.path)
201         return True
202     except (OSError, subprocess.SubprocessError):
203         return False
204
205
206 def _q(path: str) -> str:
207     if path.startswith("~/"):
208         return "~/'" + path[2:].replace("'", "'\\'") + "'"
209     return "'" + path.replace("'", "'\\'") + "'"
210
211
212 def _report(title: str, rows: List[Entry]) -> int:
213     print(f"\n{title}")
214     if not rows:
215         print(" (none)")

```

```

216         return 0
217     total = 0
218     for e in sorted(rows, key=lambda x: x.size, reverse=True):
219         cat = e.category[0]
220         print(f" {human(e.size):>8} {cat:<22} {e.path}")
221         total += e.size
222     print(f" {'-' * 8}")
223     print(f" {human(total):>8} TOTAL ({len(rows)} items)")
224     return total
225
226
227     def main(argv: Optional[List[str]] = None) -> int:
228         ap = argparse.ArgumentParser(description="Report/clean regenerable pipeline caches
(dry-run by default).")
229         ap.add_argument("--apply", action="store_true", help="actually delete (default:
report only)")
230         ap.add_argument("--keep-video", default=None, help="protect entries whose name
contains this id/substring")
231         ap.add_argument("--older-than", type=float, default=0.0, help="only purge entries
older than N days")
232         ap.add_argument("--include-wasbcache", action="store_true", help="also purge
detector resume caches")
233         ap.add_argument("--stage-dir", default="~/clips", help="WSL stage dir (default
~/clips)")
234         ap.add_argument("--distro", default="Ubuntu", help="WSL distro (default Ubuntu)")
235         ap.add_argument("--output-dir", default="output", help="Windows output dir (default
output)")
236         ap.add_argument("--summary", action="store_true",
237                         help="print a single-line nudge (only if reclaimable >= --threshold-
gb); for hooks")
238         ap.add_argument("--threshold-gb", type=float, default=5.0,
239                         help="--summary stays silent below this many reclaimable GB (default
5)")
240         args = ap.parse_args(argv)
241
242         import time
243         now = time.time()
244         entries = scan_wsl(args.distro, args.stage_dir) + scan_windows(args.output_dir)
245         to_delete, kept = plan_deletions(
246             entries, keep_video=args.keep_video, older_than_days=args.older_than,
247             include_wasbcache=args.include_wasbcache, now=now)
248
249         if args.summary:
250             reclaim = sum(e.size for e in to_delete)
251             if reclaim >= args.threshold_gb * (1024 ** 3):
252                 print(f"?? {human(reclaim)} of regenerable pipeline cache reclaimable "
253                       f"? run `python -m tools.cleanup_caches` to review, `--apply` to
delete.")
254             return 0
255
256         reclaim = _report("PURGE candidates (regenerable):", to_delete)
257         _report("KEEP (durable / protected):", kept)
258         print(f"\nReclaimable: {human(reclaim)}")
259
260         if not args.apply:
261             print("\nDRY RUN ? nothing deleted. Re-run with --apply to delete the PURGE
list.")
262             return 0
263         if not to_delete:
264             print("\nNothing to delete.")
265             return 0
266
267         print(f"\nDeleting {len(to_delete)} items...")
268         freed, failed = 0, 0
269         for e in to_delete:

```

```

270         if _delete(e, args.distro):
271             freed += e.size
272         else:
273             failed += 1
274             print(f"  FAILED: {e.path}")
275     print(f"Freed ~{human(freed)}" + (f" ({failed} failed)" if failed else ""))
276     return 0
277
278
279 if __name__ == "__main__":
280     sys.exit(main())
281

```

34. assistant (2026-06-18T01:34:39.460Z)

Note: `\_report` returns `total` (an int), but its declared return type is `int` and `reclaim = \_report(...)` uses it. That's fine. The `\_report` return for empty rows returns 0. Fine.

Let me look at `wasb\_infer.py`, `calibrate\_local.py`, `calibrate\_wasb.py` argparse, and the harness.

35. user (2026-06-18T01:34:40.339Z)

```

1      """Gen-0 TAL harness (M0.4): ONE command -> train -> honest LOVO eval -> ship-gate ->
artifact.
2
3      python -m training.gen0.harness --manifest output/human_lovo_manifest.json \
4      --floor eval_baselines/heuristic_lovo_n6.json --out artifacts/rally_tal_gen0
--version 0.1.0
5
6      Productionizes the rally_seq prototype per ADR-008:
7      - TRAINS on the corpus manifest (per-video trajectory + golden labels), evaluates with
8      honest leave-one-video-out (split by match, threshold tuned on train only), then fits
9      the FINAL model on all videos.
10     - Emits a versioned ARTIFACT BUNDLE ? the only thing serving may ever consume:
11         <out>/<version>/weights.json    (gitignored; classifier + frozen normalization)
12         <out>/<version>/manifest.json   (the reviewable contract: feature-schema version,
13                                         calibration, training lineage + GT hashes,
14                                         eval results, attestations, gate verdict)
15     - Runs the SHIP-GATE as pre-flight: floor comparison + a PAIRED per-video sign test
16       (at n=6 a mean gap alone is noise-ambiguous), plus the hard attestation blocks
17       (WASB C3 unresolved => commercial ship refused; explicit F1@tIoU target still
18       undefined => no self-declared victory). The product side owns final admission
19       (backend/eval/regression.py); this harness never grades itself into serving.
20
21     Training may import backend.* (one-way wall); backend must never import training.*.
22     """
23     from __future__ import annotations
24
25     import argparse
26     import hashlib
27     import json
28     import os
29     import subprocess
30     import time
31     from math import comb
32     from typing import Dict, List, Optional
33
34     import numpy as np
35
36     from backend.eval.classifier import LogisticRegression
37     from backend.eval.gt_loader import load_gt_intervals
38     from backend.eval.regression import gt_hash
39     from backend.eval.rally_seq_proto import (
40         labels_for, run as lovo_run, _seg_f1,

```

```

41     )
42     from backend.features.rally_seq import FEAT_NAMES, FEATURE_SCHEMA, build_frame_arrays,
features
43     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
44
45     # Post-processing constants ? part of the model's identity, recorded in the manifest.
46     CALIBRATION_DEFAULTS = {"smooth": 7, "merge_gap": 1.0, "min_dur": 1.0}
47     THRESHOLD_GRID = [round(t, 2) for t in np.arange(0.2, 0.81, 0.05)]
48
49
50     def _git_sha() -> str:
51         try:
52             return subprocess.run(["git", "rev-parse", "HEAD"], capture_output=True,
53                                   text=True, timeout=10).stdout.strip() or "unknown"
54         except Exception:
55             return "unknown"
56
57
58     def _sha256(path: str) -> str:
59         h = hashlib.sha256()
60         with open(path, "rb") as f:
61             for chunk in iter(lambda: f.read(65536), b""):
62                 h.update(chunk)
63         return h.hexdigest()
64
65
66     def sign_test(learned: List[float], floor: List[float]):
67         """Paired one-sided sign test: is learned > floor more often than chance?
68
69         At n=6 a mean comparison alone is inside noise (SE ~0.11); the paired per-video
70         wins are the defensible statistic. Returns (wins, n, p_one_sided)."""
71         pairs = [(lv, fv) for lv, fv in zip(learned, floor) if lv != fv] # ties drop,
standard
72         wins = sum(1 for lv, fv in pairs if lv > fv)
73         n = len(pairs)
74         p = sum(comb(n, k) for k in range(wins, n + 1)) / (2 ** n) if n else 1.0
75         return wins, n, p
76
77
78     def train_final(specs: List[Dict]):
79         """Fit the FINAL model on ALL corpus videos; threshold tuned on the same (train-
side).
80
81         The honest generalization estimate is the LOVO result, not this fit ? recorded as
such."""
82         vids = []
83         for s in specs:
84             pts = TrackNetRunner.parse_trajectory_csv(s["trajectory"])
85             arr = build_frame_arrays(pts, float(s["frame_width"]), float(s["fps"]))
86             X, times = features(arr, float(s["fps"]))
87             gts = load_gt_intervals(s["labels"], strict=False)
88             vids.append({"name": s["name"], "X": X, "times": times,
89                        "y": labels_for(times, gts), "gts": gts})
90         Xall = np.vstack([v["X"] for v in vids])
91         yall = np.concatenate([v["y"] for v in vids])
92         clf = LogisticRegression(lr=0.2, epochs=2000, l2=1e-2)
93         clf.fit(Xall, yall, feature_names=FEAT_NAMES)
94         thr = max(THRESHOLD_GRID, key=lambda t: _seg_f1(clf, vids, float(t)))
95         return clf, float(thr)
96
97
98     def gate_verdict(eval_res: dict, floor: Optional[dict]) -> dict:
99         """Pre-flight ship-gate.
100
101         Provisional F1 target (NOT owner-ratified; calibration tracked in #172).

```

```

102     Floor (necessary): heuristic LOVO ~0.480 (from quality table).
103     Multi-court ref (makes owned \"worth serving\" per data): 0.66.
104     Proposed for \"very high P/R\" + significance: F1@tIoU0.5 >= 0.70 on
105     multi-court folds with paired-sign p<0.05 vs heuristic (refine as corpus grows).
106     Beating floor + sign test necessary; C3 main commercial blocker.
107     ship=True only when all non-C3 blocks clear (honesty first).
108     """
109     reasons: List[str] = []
110     floor_passed = None
111     sign = None
112     if floor:
113         floor_mean = float(floor["mean_f1"])
114         floor_passed = eval_res["mean_f1"] > floor_mean
115         if not floor_passed:
116             reasons.append(f"LOVO mean {eval_res['mean_f1']:.3f} <= floor
117 {floor_mean:.3f}")
118             per = floor.get("per_video", {})
119             paired = [(r["f1"], per[r["video"]]) for r in eval_res["per_video"] if
120 r["video"] in per]
121             if paired:
122                 wins, n, p = sign_test([lv for lv, _ in paired], [fv for _, fv in paired])
123                 sign = {"wins": wins, "n": n, "p_one_sided": round(p, 4)}
124                 if p >= 0.05:
125                     reasons.append(f"paired sign test not significant ({wins}/{n} wins,
126 p={p:.3f}) \"
127                                     f\"? grow the corpus before claiming superiority")
128             else:
129                 reasons.append("no floor baseline provided ? nothing to beat")
130
131     # Provisional blocker (per owner review on #170; see #172 for calibration).
132     # (The number itself is not the blocker; calibration tracked in #172.)
133     reasons.append("ship-gate F1@tIoU target NOT YET CALIBRATED ? provisional; see issue
134 #172")
135     reasons.append("WASB checkpoint provenance C3 UNRESOLVED ? commercial ship blocked
136 regardless of F1")
137     # ship remains False until C3 resolved + other gates (by design).
138     return {"ship": False, "floor_passed": floor_passed, "sign_test": sign, "blockers":
139 reasons}
140
141 def run(manifest_path: str, out_dir: str, version: str,
142         floor_path: Optional[str] = None) -> dict:
143     with open(manifest_path, encoding="utf-8") as f:
144         specs = json.load(f)
145         floor = None
146         if floor_path:
147             with open(floor_path, encoding="utf-8") as f:
148                 floor = json.load(f)
149
150     # 1) Honest LOVO eval (the number that matters).
151     eval_res = lovo_run(specs)
152
153     # 2) Final model on all videos.
154     clf, thr = train_final(specs)
155
156     # 3) Gate (pre-flight).
157     verdict = gate_verdict(eval_res, floor)
158
159     # 4) Artifact bundle.
160     bundle_dir = os.path.join(out_dir, version)
161     os.makedirs(bundle_dir, exist_ok=True)
162     weights_path = os.path.join(bundle_dir, "weights.json")
163     payload = clf.to_dict()
164     payload["calibration"] = {"threshold": thr, **CALIBRATION_DEFAULTS}
165     payload["feature_schema_version"] = FEATURE_SCHEMA["version"]

```



```

161     with open(weights_path, "w", encoding="utf-8") as f:
162         json.dump(payload, f, indent=2)
163
164     manifest = {
165         "artifact": {
166             "name": "rally_tal_gen0", "version": version,
167             "created_at": time.strftime("%Y-%m-%dT%H:%M:%S%z"),
168             "git_sha": _git_sha(),
169             "weights_file": "weights.json", "weights_sha256": _sha256(weights_path),
170         },
171         "feature_schema": FEATURE_SCHEMA,
172         "calibration": {"threshold": thr, **CALIBRATION_DEFAULTS,
173             "note": "threshold tuned train-side on the full corpus; "
174             "honest generalization = the LOVO eval below"},
175         "training_lineage": {
176             "corpus_manifest": manifest_path,
177             "split_protocol": "leave-one-video-out by match",
178             "label_source": "human (rally-annotator golden CSVs)",
179             "videos": [{"name": s["name"], "labels": s["labels"],
180                 "gt_hash": gt_hash(load_gt_intervals(s["labels"],
181 strict=False)),
182                 "fps": s["fps"]} for s in specs],
183         },
184         "eval": {
185             "protocol": "LOVO, F1@tIoU=0.5 segment-level + per-frame AUC",
186             "per_video": eval_res["per_video"],
187             "mean_f1": eval_res["mean_f1"],
188             "floor": floor,
189         },
190         "attestation": {
191             "deps_licenses": "numpy/scipy (BSD) + repo-internal code only (Gen-0)",
192             "wasb_checkpoint_provenance": "C3-UNRESOLVED (ship-blocking; see
193 COMMERCIALIZATION C3)",
194             "no_paid_llm_training_labels": True,
195             "consent_ledger": "pending-M0.1 (owner-attested: owned footage, all-adult
196 sessions)",
197         },
198         "gate": verdict,
199     }
200
201     manifest_out = os.path.join(bundle_dir, "manifest.json")
202     with open(manifest_out, "w", encoding="utf-8") as f:
203         json.dump(manifest, f, indent=2)
204
205     print(f"\n=== Gen-0 artifact bundle: {bundle_dir} ===")
206     print(f"weights : {weights_path}")
207     print(f"manifest: {manifest_out}")
208     print(f"GATE      : ship={verdict['ship']} floor_passed={verdict['floor_passed']} "
209           f"sign_test={verdict['sign_test']}")
210     for b in verdict["blockers"]:
211         print(f"    - {b}")
212     return manifest
213
214 def main() -> None:
215     ap = argparse.ArgumentParser(description=__doc__)
216     ap.add_argument("--manifest", required=True,
217                     help="corpus manifest JSON:
218 [{name,trajectory,fps,frame_width,labels}, ...]")
219     ap.add_argument("--floor", default=None,
220                     help="floor baseline JSON (e.g.
221 eval_baselines/heuristic_lovo_n6.json)")
222     ap.add_argument("--out", default=os.path.join("artifacts", "rally_tal_gen0"))
223     ap.add_argument("--version", required=True, help="semver for the artifact bundle")
224     args = ap.parse_args()
225     run(args.manifest, args.out, args.version, floor_path=args.floor)

```

```

221
222
223     if __name__ == "__main__":
224         main()
225

```

36. assistant (2026-06-18T01:34:45.940Z)

This is mostly fine. Let me look at the remaining argparse files quickly: `wasb\_infer.py`, `calibrate\_local.py`, `calibrate\_wasb.py`, `export\_wasb\_segments.py`, `distill\_local.py`, `eval\_partial\_labels.py`, `gemini\_refine.py`, `served\_gate\_a.py`, `serve\_contrast.py`, `rally\_seg\_eval.py`, `golden\_fixtures.py`.

Let me prioritize the ones most likely to have request-handling/argparse bugs and check the calibrate scripts which often have threshold-sweep off-by-ones.

37. user (2026-06-18T01:34:46.929Z)

```

1     """Distill Gemini (teacher) into the LOCAL-ONLY pipeline (student) ? calibrate the
2     on-device WASB+heuristic knobs to match Gemini's rally labels, so production runs
3     100% local (no cloud, privacy lever intact, ~zero marginal cost).
4
5     Pipeline being tuned (all local, no Gemini at runtime):
6         WASB trajectory CSV -> trajectory_to_action_windows(velocity, merge_gap, min_dur)
7                               -> rally_gate (min_crossings)                -> rally windows
8
9     Ground truth = Gemini full-context silver labels (one CSV per video). With >=2
10    videos we use leave-one-video-out (tune on the others, score the held-out one) ?
11    the honest "how precise is the local pipeline on an unseen video" number. The
12    output is a single local config + the precision it buys, with NO Gemini dependency
13    shipped.
14
15    Reuses: TrackNetRunner windowing, rally_gate, calibrate_wasb.load_golden_gts,
16    calibration.{select_best, leave_one_video_out, improvement_report}.
17
18    Run:
19        python -m backend.eval.calibrate_local --manifest videos.json
20    where videos.json = [{"name","trajectory","fps","frame_width","labels"}, ...]
21    (labels = a Gemini full-context CSV; trajectory = the WASB CSV).
22    """
23    from __future__ import annotations
24
25    import json
26    import argparse
27    from typing import Callable, Dict, List, Tuple
28
29    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
30    from backend.pipeline.detectors import rally_gate
31    from backend.eval.calibrate_wasb import load_golden_gts
32    from backend.eval.calibration import select_best, leave_one_video_out,
improvement_report, evaluate
33
34    # (velocity_thresh, merge_gap, min_window_duration, min_crossings)
35    LocalConfig = Tuple[float, float, float, int]
36    BASELINE_LOCAL: LocalConfig = (0.02, 2.0, 2.0, 0) # today's local default (windowing,
no gate)
37
38
39    def make_local_predict_fn(trajectory_csv: str, fps: float, frame_width: float) ->
Callable[[LocalConfig], list]:
40        """predict_fn(cfg) -> rally windows from a cached WASB trajectory, LOCAL ONLY
41        (windowing thresholds + the shuttle net-crossing gate). Parsed once."""
42        points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
43
44        def predict_fn(cfg: LocalConfig) -> List[Tuple[float, float]]:

```

```

45         vt, mg, mwd, mc = cfg
46         windows = TrackNetRunner.trajectory_to_action_windows(
47             points, fps=fps, frame_width=frame_width,
48             velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
49         )
50         wins = [(w["start"], w["end"]) for w in windows]
51         if mc > 0:
52             wins = rally_gate.apply_gate(points, wins, fps, min_crossings=mc)
53         return wins
54
55     return predict_fn
56
57
58     def local_grid() -> List[LocalConfig]:
59         return [(vt, mg, mwd, mc)
60                 for vt in (0.005, 0.01, 0.02, 0.03, 0.05)
61                 for mg in (1.0, 2.0, 3.0)
62                 for mwd in (1.0, 2.0, 3.0)
63                 for mc in (0, 1, 2, 3)]
64
65
66     def build_videos(specs: List[Dict]) -> List[Dict]:
67         videos = []
68         for s in specs:
69             pf = make_local_predict_fn(s["trajectory"], float(s["fps"]),
70 float(s["frame_width"]))
71             gts = load_golden_gts(s["labels"])
72             if not gts:
73                 raise SystemExit(f"no usable labels in {s['labels']} for {s.get('name')}")
74             videos.append({"name": s.get("name", s["trajectory"]), "predict_fn": pf, "gts":
75 gts})
76
77         return videos
78
79
80     def run(specs: List[Dict], iou: float = 0.5, metric: str = "f1") -> dict:
81         videos = build_videos(specs)
82         grid = local_grid()
83         print(f"=== Local-only calibration vs Gemini silver-GT ({len(videos)} videos, "
84 f"{len(grid)} configs, IoU>={iou}, optimise {metric}) ===")
85
86         # Per-video: today's baseline vs the best-fit local config (optimistic ? fit on that
87 video).
88         per_video = []
89         for v in videos:
90             best_cfg, best_val = select_best(v["predict_fn"], grid, v["gts"], metric, iou)
91             imp = improvement_report(v["predict_fn"], BASELINE_LOCAL, best_cfg, v["gts"],
92 iou)
93
94             b, t = imp["baseline"], imp["tuned"]
95             print(f"\n[{v['name']}] {len(v['gts'])} Gemini rallies")
96             print(f"  baseline {BASELINE_LOCAL}: P={b['precision']:.3f} R={b['recall']:.3f}
97 F1={b['f1']:.3f}")
98             print(f"  best-fit {best_cfg}: P={t['precision']:.3f} R={t['recall']:.3f}
99 F1={t['f1']:.3f} "
100 f"(F1 {imp['f1_delta_pct']:+.0f}%, fit-on-self = optimistic)")
101             per_video.append({"name": v["name"], "best_cfg": best_cfg, "improvement": imp})
102
103         # Honest cross-video: pick config on the OTHER videos, score the held-out one.
104         lovo = leave_one_video_out(videos, grid, metric, iou)
105         # Baseline's held-out score (no tuning) for the honest before/after.
106         base_held = [evaluate(v["predict_fn"](BASELINE_LOCAL), v["gts"], iou) for v in
107 videos]
108         base_mean = sum(getattr(s, metric) for s in base_held) / len(base_held)
109
110         print("\n--- HONEST leave-one-video-out (the number to trust) ---")
111         if "error" in lovo:

```

```

103         print(f"  {lovo['error']} (need >=2 videos)")
104     else:
105         print(f"  baseline (untuned) mean held-out {metric}: {base_mean:.3f}")
106         print(f"  calibrated mean held-out {metric}:          {lovo['mean_held_out']:.3f}")
107         f"± {lovo['std_held_out']:.3f} (gap {lovo['generalization_gap']:+.3f})"
108         for p in lovo["per_video"]:
109             print(f"    held-out {p['video']}: {p['held_out']:.3f} (cfg picked on
others: {p['config']}")
110         return {"per_video": per_video, "lovo": lovo, "baseline_held_out": base_mean,
"metric": metric}
111
112
113     def main() -> None:
114         ap = argparse.ArgumentParser(description="Calibrate the local-only pipeline against
Gemini silver labels.")
115         ap.add_argument("--manifest", required=True,
116                         help="JSON: [{name, trajectory, fps, frame_width, labels}, ...]")
117         ap.add_argument("--iou", type=float, default=0.5)
118         ap.add_argument("--metric", default="f1", choices=["f1", "precision", "recall"])
119         args = ap.parse_args()
120         with open(args.manifest, encoding="utf-8") as f:
121             specs = json.load(f)
122         run(specs, iou=args.iou, metric=args.metric)
123
124
125     if __name__ == "__main__":
126         main()
127

```

38. assistant (2026-06-18T01:34:51.152Z)

Line 99: ``base_mean = sum(...) / len(base_held)`` ? if ``base_held`` is empty (empty manifest), this is a ``ZeroDivisionError``. But ``build_videos`` would return an empty list for empty specs, and the grid is non-empty. This is an edge case but the manifest is required and typically non-empty. Low severity, likely won't be hit. Let me move on.

Let me look at ``wasb_infer.py``, ``served_gate_a.py``, ``serve_contrast.py`` which are FastAPI-ish (serve names).

39. user (2026-06-18T01:34:52.151Z)

```

1      """SERVED Gate-A verification: run the *production* ``FusionSegmenter`` decision seems
2      on a cached WASB trajectory and score the resulting AI-handoff windows vs golden labels.
3
4      The offline litmus (``backend.eval.ablation`` / ``fusion_compare``) measures Gate A on
the
5      **permissive proposal windowing** (``distill_local.PROPOSAL`` = recall-oriented) re-
scored
6      through the experiment harness's ``_gate_mask``. That validates the *signal*. It does
NOT
7      exercise the **served path**: in production the windows come from
8      ``trajectory_to_action_windows`` at the production operating point
9      (``velocity_threshold=0.02, dead_time_merge_gap=2.0, min_action_window_duration=2.0``),
the
10     net-crossing count is computed by ``FusionSegmenter._enrich_candidate_stats``, and the
gate is
11     ``FusionSegmenter._select_for_handoff``. THIS module drives exactly those production
methods ?
12     no reimplementaion of the windowing or the gate ? on a cached trajectory CSV, so the
13     verification reflects what ``process_video`` actually hands to Gemini.
14
15     GPU-free and Gemini-free: it stops at the handoff-selection boundary (the windows that
WOULD
16     be sent to the AI). With the person cue OFF (the default) nothing here loads torch or

```

```

touches
17     the GPU ? it just re-scores cached WASB trajectories.
18
19     Run (owner box, golden trajectories present):
20     python -m backend.eval.served_gate_a --manifest output/human_lovo_manifest.json
21     """
22     from __future__ import annotations
23
24     import argparse
25     import json
26     from dataclasses import dataclass
27     from typing import Any, Dict, List, Optional, Tuple
28
29     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
30     from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
31     from backend.eval.calibrate_wasb import load_golden_gts
32     from backend.eval.metrics import Interval, score, Score
33     from backend.eval import segmentation_metrics as _segm
34
35     # Production windowing operating point (config.json indexing defaults ? the values
36     # TrajectoryHybridSegmenter.process_video reads for the served fusion run).
37     PROD_VELOCITY_THRESH = 0.02
38     PROD_MERGE_GAP = 2.0
39     PROD_MIN_WINDOW = 2.0
40
41
42     def _served_idx_cfg(min_crossings: int) -> Dict[str, Any]:
43         """The ``indexing`` config dict the served FusionSegmenter seams read ? mirrors
44         config.json's ``indexing.fusion`` block, with Gate A at ``min_crossings`` and the
45         person cue OFF (so no GPU / torch is touched)."""
46         return {
47             "fusion": {
48                 "substrate": "wasb",
49                 "min_crossings": min_crossings,
50                 "min_players": 0,
51                 "cue_flags": {},
52             },
53             "wasb": {"velocity_threshold": PROD_VELOCITY_THRESH},
54             "dead_time_merge_gap": PROD_MERGE_GAP,
55             "min_action_window_duration": PROD_MIN_WINDOW,
56         }
57
58
59     def served_handoff_windows(trajecory_csv: str, fps: float, frame_width: float,
60                               min_crossings: int) -> List[Interval]:
61         """Run the PRODUCTION FusionSegmenter decision path on a cached trajectory and
62         return the
63         windows that would reach the AI handoff (i.e. the served rally candidates).
64         Uses the real ``FusionSegmenter`` seam methods ? ``_enrich_candidate_stats``
65         (computes ``net_crossings``) and ``_select_for_handoff`` (Gate A) ? over windows from the
66         production
67         ``trajectory_to_action_windows`` operating point. No GPU, no Gemini: the person cue
68         is OFF.
69         """
70         points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
71         windows = TrackNetRunner.trajectory_to_action_windows(
72             points, fps=fps, frame_width=frame_width,
73             velocity_thresh=PROD_VELOCITY_THRESH, merge_gap=PROD_MERGE_GAP,
74             min_window_duration=PROD_MIN_WINDOW,
75         )
76         idx_cfg = _served_idx_cfg(min_crossings)
77         # Construct the real served segmenter (db=None, config carries the indexing block).
78         We only

```

```

76         # call the pure decision seems ? no process_video, no DB, no provider ? exactly as
the
77         # registry constructs it for `name`/`description`.
78         seg = FusionSegmenter(None, {"indexing": idx_cfg, "sport": "badminton"})
79         # The served per-window stats (incl. net_crossings) via the production enrichment
hook.
80         window_stats = [
81             seg._enrich_candidate_stats(cw, points, fps, frame_width, video_path="",
82                                         idx_cfg=idx_cfg)
83             for cw in windows
84         ]
85         keep = seg._select_for_handoff(windows, window_stats, idx_cfg)
86         return [(windows[i]["start"], windows[i]["end"]) for i in keep]
87
88
89 @dataclass
90 class ServedVideoResult:
91     name: str
92     num_gt: int
93     on: Score          # Gate-A ON (min_crossings=2)
94     off: Score         # Gate-A OFF (min_crossings=0, control)
95     seg_ratio_on: float # |preds| / |gts| with Gate-A ON (over-segmentation)
96     seg_ratio_off: float
97     n_handoff_on: int
98     n_handoff_off: int
99
100     @property
101     def delta_f1(self) -> float:
102         return self.on.f1 - self.off.f1
103
104
105 def evaluate_video(spec: Dict[str, Any], iou: float = 0.5,
106                  on_crossings: int = 2) -> ServedVideoResult:
107     """Score the served Gate-A ON (min_crossings=on_crossings) vs OFF (0) handoff
windows for
108     one golden video against its golden labels."""
109     fps, fw = float(spec["fps"]), float(spec["frame_width"])
110     traj, labels = str(spec["trajectory"]), str(spec["labels"])
111     gts = load_golden_gts(labels)
112     preds_on = served_handoff_windows(traj, fps, fw, min_crossings=on_crossings)
113     preds_off = served_handoff_windows(traj, fps, fw, min_crossings=0)
114     return ServedVideoResult(
115         name=str(spec["name"]),
116         num_gt=len(gts),
117         on=score(preds_on, gts, iou),
118         off=score(preds_off, gts, iou),
119         seg_ratio_on=_segm.segment_count_ratio(preds_on, gts),
120         seg_ratio_off=_segm.segment_count_ratio(preds_off, gts),
121         n_handoff_on=len(preds_on),
122         n_handoff_off=len(preds_off),
123     )
124
125
126 def evaluate_manifest(specs: List[Dict[str, Any]], iou: float = 0.5,
127                    on_crossings: int = 2) -> List[ServedVideoResult]:
128     return [evaluate_video(s, iou, on_crossings) for s in specs]
129
130
131 def aggregate(results: List[ServedVideoResult]) -> Dict[str, Any]:
132     """Mean ON/OFF F1 + over-seg ratio, the paired ?F1, and the per-video win count."""
133     n = len(results)
134
135     def _mean(getter) -> float:
136         return sum(getter(r) for r in results) / n if n else 0.0
137

```

```

138     deltas = [r.delta_f1 for r in results]
139     wins = sum(1 for d in deltas if d > 1e-9)
140     losses = sum(1 for d in deltas if d < -1e-9)
141     return {
142         "num_videos": n,
143         "mean_f1_on": _mean(lambda r: r.on.f1),
144         "mean_f1_off": _mean(lambda r: r.off.f1),
145         "mean_delta_f1": _mean(lambda r: r.delta_f1),
146         "mean_precision_on": _mean(lambda r: r.on.precision),
147         "mean_precision_off": _mean(lambda r: r.off.precision),
148         "mean_recall_on": _mean(lambda r: r.on.recall),
149         "mean_recall_off": _mean(lambda r: r.off.recall),
150         "mean_seg_ratio_on": _mean(lambda r: r.seg_ratio_on),
151         "mean_seg_ratio_off": _mean(lambda r: r.seg_ratio_off),
152         "wins": wins,
153         "losses": losses,
154     }
155
156
157 def format_table(results: List[ServedVideoResult], agg: Dict[str, Any], iou: float) ->
str:
158     lines = [
159         f"=== SERVED Gate-A verification (production FusionSegmenter path, IoU>={iou})
===",
160         f"  {'video':<24s} {'gt':>3s} {'F1_off':>7s} {'F1_on':>7s} {'?F1':>7s} "
161         f"{'segR_off':>9s} {'segR_on':>8s} {'hand_off':>9s} {'hand_on':>8s}",
162     ]
163     for r in results:
164         lines.append(
165             f"  {r.name:<24s} {r.num_gt:>3d} {r.off.f1:>7.3f} {r.on.f1:>7.3f} "
166             f"{r.delta_f1:>+7.3f} {r.seg_ratio_off:>9.2f} {r.seg_ratio_on:>8.2f} "
167             f"{r.n_handoff_off:>9d} {r.n_handoff_on:>8d}")
168     lines += [
169         f"  {'-' * 92}",
170         f"  MEAN  F1 {agg['mean_f1_off']:.3f} (off) -> {agg['mean_f1_on']:.3f} (on) "
171         f"?F1={agg['mean_delta_f1']:+.3f} "
172         f"P {agg['mean_precision_off']:.3f}->{agg['mean_precision_on']:.3f} "
173         f"R {agg['mean_recall_off']:.3f}->{agg['mean_recall_on']:.3f}",
174         f"  over-seg ratio {agg['mean_seg_ratio_off']:.2f} (off) -> "
175         f"{agg['mean_seg_ratio_on']:.2f} (on) | wins {agg['wins']}/{agg['num_videos']}
"
176         f"(losses {agg['losses']})",
177     ]
178     return "\n".join(lines)
179
180
181 def run(manifest_path: str, iou: float = 0.5) -> Tuple[List[ServedVideoResult],
Dict[str, Any]]:
182     with open(manifest_path, encoding="utf-8") as f:
183         specs = json.load(f)
184         results = evaluate_manifest(specs, iou=iou)
185         return results, aggregate(results)
186
187
188 def main(argv: Optional[List[str]] = None) -> None:
189     ap = argparse.ArgumentParser(
190         description="Verify served Gate-A on real footage via the production
FusionSegmenter path.")
191     ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
192     ap.add_argument("--iou", type=float, default=0.5)
193     args = ap.parse_args(argv)
194     results, agg = run(args.manifest, iou=args.iou)
195     print(format_table(results, agg, args.iou))
196
197

```

```
198     if __name__ == "__main__":
199         main()
200
```

40. assistant (2026-06-18T01:34:56.061Z)

This is clean. Let me verify my primary finding by checking the test for `run\_phase3\_eval.py` and confirming `Success` has no `\_\_len\_\_`. Let me also re-examine the `process\_video` signature in `batch.py` ? actually the eval driver calls `process\_video` directly. Let me confirm the motion segmenter is registered with key "motion" and is the default.

41. user (2026-06-18T01:34:59.203Z)

```
14:         return "motion"
227:segmenter_registry.register(MotionPlaySegmenter)
---tests for phase3---
tests/test_api_endpoints.py
tests/test_async_jobs.py
tests/test_base.py
tests/test_fusion_hybrid.py
tests/test_gemini.py
tests/test_generate_highlights.py
tests/test_job_worker_loop.py
tests/test_tracknet_hybrid.py
tests/test_trajectory_hybrid_equivalence.py
tests/test_windowing.py
```

42. assistant (2026-06-18T01:35:03.733Z)

Confirmed: `MotionPlaySegmenter` returns `"motion"` as name and is registered. The default `--segmenter` in `run\_phase3\_eval.py` is `motion`. There's no test covering the `len(segs)` path with the motion segmenter, so this bug is uncaught.

Let me verify there's no `\_\_len\_\_` defined on Success via the dataclass ? confirmed from results.py (plain dataclass, no `\_\_len\_\_`). So `len(Success(...))` ? `TypeError`.

Now let me confirm the exact line numbers and check the motion segmenter's full process_video to be 100% sure it returns Success in the normal path.

43. user (2026-06-18T01:35:04.793Z)

```
218             "duration": dur,
219             "server": server,
220             "receiver": receiver,
221             "metadata": metadata
222         })
223
224         return Success(value=segments_data)
225
226     # Register the segmenter
227     segmenter_registry.register(MotionPlaySegmenter)
```

44. assistant (2026-06-18T01:35:10.006Z)

Confirmed. `MotionPlaySegmenter.process\_video` returns `Success(value=segments\_data)` on the normal path, and `Failure(...)` if the video can't be opened. Both lack `\_\_len\_\_`.

So in `scripts/run\_phase3\_eval.py` line 146, `len(segs)` raises `TypeError` whenever the motion segmenter (the default) processes a video. The documented "Free CPU baseline" example (`--segmenter motion`) crashes.

Let me do a final scan of the remaining argparse scripts to make sure I'm not missing other clear-cut bugs in this class. Let me quickly check `calibrate\_wasb.py`, `distill\_local.py`, `export\_wasb\_segments.py`, `eval\_partial\_labels.py`, `gemini\_refine.py`, `serve\_contrast.py`,

`rally\_seg\_eval.py`, `wasb\_infer.py`, `golden\_fixtures.py` for argparse/coercion issues ?
focusing on the `main()/argparse` sections.

45. user (2026-06-18T01:35:13.892Z)

```
==== backend/eval/calibrate_wasb.py ====
97:  ap.add_argument("--trajectory", required=True, help="WASB trajectory CSV
(Frame,Visibility,X,Y)")
98:  ap.add_argument("--labels", required=True, help="golden labels CSV (start_time/end_time
cols)")
99:  ap.add_argument("--fps", type=float, default=59.94)
100: ap.add_argument("--frame-width", type=float, default=1920.0)
101: ap.add_argument("--iou", type=float, default=0.5)
102: args = ap.parse_args()
103: run(args.trajectory, args.labels, args.fps, args.frame_width, args.iou)
==== backend/eval/distill_local.py ====
153: ap.add_argument("--manifest", required=True,
155: ap.add_argument("--iou", type=float, default=0.5)
156: ap.add_argument("--threshold", type=float, default=0.5, help="classifier probability
cutoff")
157: ap.add_argument("--model-out", default="output/local_rally_model.json")
158: args = ap.parse_args()
159: with open(args.manifest, encoding="utf-8") as f:
161: run(specs, iou=args.iou, threshold=args.threshold, model_out=args.model_out)
==== backend/eval/export_wasb_segments.py ====
152: print(f"[gate] {label}: {len(segs)} -> {len(kept)} segments "
153:       f"(removed {len(segs) - len(kept)} with <{min_crossings} net-crossings)")
200: ap.add_argument("--trajectory", required=True, help="WASB trajectory CSV
(Frame,Visibility,X,Y)")
201: ap.add_argument("--fps", type=float, default=59.94)
202: ap.add_argument("--frame-width", type=float, default=1920.0)
203: ap.add_argument("--velocity-thresh", type=float, default=TUNED[0])
204: ap.add_argument("--merge-gap", type=float, default=TUNED[1])
205: ap.add_argument("--min-duration", type=float, default=TUNED[2])
206: ap.add_argument("--labels", default=None, help="golden labels CSV -> tuned-vs-golden
comparison")
207: ap.add_argument("--iou", type=float, default=0.5)
208: ap.add_argument("--out", default="output/wasb_tuned_segments.csv")
209: ap.add_argument("--comparison-out", default=None)
210: ap.add_argument("--baseline", action="store_true", help="also export baseline-config
segments")
211: ap.add_argument("--min-crossings", type=int, default=0,
214: ap.add_argument("--video", default=None, help="source video (with --slice) to cut clips
from")
215: ap.add_argument("--slice", action="store_true", help="cut one padded clip per tuned
rally")
216: ap.add_argument("--clips-dir", default="output/wasb_tuned_clips")
217: args = ap.parse_args()
219: cfg = (args.velocity_thresh, args.merge_gap, args.min_duration)
220: res = run(args.trajectory, args.fps, args.frame_width, cfg=cfg, out_csv=args.out,
221:          labels_csv=args.labels, iou=args.iou, baseline=args.baseline,
222:          comparison_out=args.comparison_out, min_crossings=args.min_crossings)
223: if args.slice:
224:     if not args.video:
226:         _maybe_slice(args.video, res["out_csv"], args.clips_dir)
==== backend/eval/eval_partial_labels.py ====
193: ap.add_argument("--trajectory", required=True, help="WASB trajectory CSV
(Frame,Visibility,X,Y)")
194: ap.add_argument("--labels", required=True, help="human golden CSV
(rally_number,start_time,end_time,...)")
195: ap.add_argument("--fps", type=float, default=59.94)
196: ap.add_argument("--frame-width", type=float, default=1920.0)
197: ap.add_argument("--window-start", type=float, default=0.0)
198: ap.add_argument("--window-end", type=float, default=None,
200: ap.add_argument("--gate", type=int, default=2, help="net-crossing gate K (0=off, 2
```

```

recommended)")
201:     ap.add_argument("--iou", type=float, default=0.5)
202:     ap.add_argument("--comparison-out", default=None)
203:     ap.add_argument("--sweep", action="store_true",
205:     ap.add_argument("--top", type=int, default=12, help="how many sweep rows to print")
206:     args = ap.parse_args()
207:     if args.sweep:
208:         sweep(args.trajectory, args.labels, args.fps, args.frame_width,
209:             window_start=args.window_start, window_end=args.window_end,
210:             iou=args.iou, top=args.top)
212:         run(args.trajectory, args.labels, args.fps, args.frame_width,
213:             window_start=args.window_start, window_end=args.window_end,
214:             gate_crossings=args.gate, iou=args.iou, comparison_out=args.comparison_out)
===== backend/eval/gemini_refine.py =====
201:     ap.add_argument("--video", required=True)
202:     ap.add_argument("--trajectory", default=None, help="WASB trajectory CSV (required unless
--full-video)")
203:     ap.add_argument("--fps", type=float, default=59.94)
204:     ap.add_argument("--frame-width", type=float, default=1920.0)
205:     ap.add_argument("--config", choices=["baseline", "tuned"], default="baseline",
207:     ap.add_argument("--pad", type=float, default=3.0)
208:     ap.add_argument("--model", default="gemini-flash-latest")
209:     ap.add_argument("--max-workers", type=int, default=4)
210:     ap.add_argument("--min-dur", type=float, default=2.0)
211:     ap.add_argument("--out", default="output/gemini_refined.csv")
212:     ap.add_argument("--limit", type=int, default=0, help="only the first N candidates (smoke
test)")
213:     ap.add_argument("--clip-scale-width", type=int, default=1280,
215:     ap.add_argument("--full-video", action="store_true",
218:     args = ap.parse_args()
219:     scale = args.clip_scale_width or None
220:     if args.full_video:
221:         full_video_rallies(args.video, model=args.model, out_csv=args.out,
222:             clip_scale_width=scale, min_dur=args.min_dur,
max_workers=args.max_workers)
224:     if not args.trajectory:
226:         cfg = BASELINE if args.config == "baseline" else TUNED
227:         run(args.video, args.trajectory, args.fps, args.frame_width, cfg=cfg, pad=args.pad,
228:             model=args.model, max_workers=args.max_workers, out_csv=args.out,
229:             min_dur=args.min_dur, limit=args.limit, clip_scale_width=scale)
===== backend/eval/serve_contrast.py =====
145:     ap.add_argument("--manifest", default="output/human_lovo_manifest.json",
147:     ap.add_argument("--iou", type=float, default=IOU)
148:     ap.add_argument("--min-crossings", type=int, default=MIN_CROSSINGS)
149:     ap.add_argument("--min-n", type=int, default=5, help="promotion min-N floor")
150:     ap.add_argument("--served-regress-eps", type=float, default=0.0,
152:     ap.add_argument("--json", default=None, help="also write the structured result JSON
here")
153:     args = ap.parse_args()
155:     with open(args.manifest, encoding="utf-8") as f:
157:         result = run_contrast(specs, iou=args.iou, min_crossings=args.min_crossings,
158:             min_n=args.min_n, served_regress_eps=args.served_regress_eps)
160:     if args.json:
161:         with open(args.json, "w", encoding="utf-8") as f:
163:             print(f"\nwrote {args.json}")
===== backend/eval/rally_seg_eval.py =====
269:     ap.add_argument("--manifest", default=DEFAULT_MANIFEST)
270:     ap.add_argument("--features-dir", default=DEFAULT_FEATURES_DIR)
271:     ap.add_argument("--preset", default=SERVED.name, help="windowing preset (default:
served)")
272:     ap.add_argument("--vs", default=None, help="second preset to diff against --preset")
273:     ap.add_argument("--iou", type=float, default=DEFAULT_IOU)
274:     ap.add_argument("--min-crossings", type=int, default=0,
276:     ap.add_argument("--vs-min-crossings", type=int, default=0,
278:     ap.add_argument("--max-window-duration", type=float, default=None,

```

```

280:     ap.add_argument("--vs-max-window-duration", type=float, default=None,
283:     ap.add_argument("--hard-cap", action="store_true",
286:     ap.add_argument("--vs-hard-cap", action="store_true",
288:     ap.add_argument("--blob-fallback", action="store_true",
291:     ap.add_argument("--vs-blob-fallback", action="store_true",
293:     ap.add_argument("--blob-factor", type=float, default=None,
295:     ap.add_argument("--inactivity-end", type=float, default=None,
297:     ap.add_argument("--vs-inactivity-end", type=float, default=None,
299:     ap.add_argument("--inactivity-rally-thresh", type=float, default=None,
301:     ap.add_argument("--inactivity-min-density", type=float, default=None,
303:     ap.add_argument("--tau-start", type=float, default=None,
305:     ap.add_argument("--tau-end", type=float, default=None,
307:     ap.add_argument("--guard-weight-merge", type=float, default=None,
309:     ap.add_argument("--guard-weight-split", type=float, default=None,
311:     ap.add_argument("--guard-counts", action="store_true",
314:     ap.add_argument("--json-out", default=None, help="optional path to write the full result
JSON")
315:     args = ap.parse_args()
318:     golden = load_golden(args.manifest, args.features_dir)
322:         if args.inactivity_rally_thresh is not None:
323:             preset = replace(preset, inactivity_rally_thresh=args.inactivity_rally_thresh)
324:         if args.inactivity_min_density is not None:
325:             preset = replace(preset, inactivity_min_density=args.inactivity_min_density)
328:     base = _resolve_preset(args.preset)
329:     if args.max_window_duration is not None:
330:         base = replace(base, max_window_duration=args.max_window_duration)
331:     if args.hard_cap:
333:     if args.blob_fallback:
335:     if args.blob_factor is not None:
336:         base = replace(base, blob_factor=args.blob_factor)
337:     base = _apply_inactivity(base, args.inactivity_end)
338:     if args.vs:
339:         vs_mc = args.vs_min_crossings or args.min_crossings
===== backend/eval/golden_fixtures.py =====
1023:     g.add_argument("--freeze-synthetic", action="store_true",
1026:     g.add_argument("--refresh-snapshot", action="store_true",
1028:     g.add_argument("--check", action="store_true",
1030:     g.add_argument("--freeze-real", action="store_true",
1033:     ap.add_argument("--slug", help="single slug for --refresh-snapshot, or the surrogate
slug for "
1035:     ap.add_argument("--all", action="store_true", help="all slugs for --refresh-snapshot")
1036:     ap.add_argument("--reason", help="REQUIRED recorded reason for --refresh-snapshot
(review the diff!)")
1038:     ap.add_argument("--trajectory", help="--freeze-real) source trajectory CSV
(Frame,Visibility,X,Y)")
1039:     ap.add_argument("--labels", help="--freeze-real) source golden labels CSV
(start,end)")
1040:     ap.add_argument("--license", dest="license_", default=None,
1042:     ap.add_argument("--owned", action="store_true",
1044:     ap.add_argument("--minors-free", dest="minors_free", action="store_true",
1046:     ap.add_argument("--fps", type=float, help="--freeze-real) source fps")
1047:     ap.add_argument("--frame-width", dest="frame_width", type=float, help="--freeze-real)
source frame width (px)")
1048:     ap.add_argument("--label", default="cleared_real",
1051:     ap.add_argument("--source-note", dest="source_note", default="",
1055:     ap.add_argument("--no-time-origin-reset", dest="time_origin_reset",
action="store_false",
1058:     ap.add_argument("--fixtures-dir", type=Path, default=FIXTURES_DIR,
1060:     args = ap.parse_args(list(argv) if argv is not None else None)
1062:     fixtures_dir: Path = args.fixtures_dir
1064:     if args.freeze_synthetic:
1070:     if args.refresh_snapshot:
1071:         if not args.slug and not args.all:
1075:         if not (args.reason and args.reason.strip()):
1078:         targets = refresh_snapshot(slug=args.slug, all_slugs=args.all,

```

```

fixtures_dir=fixtures_dir)
1080:         f" (reason: {args.reason})")
1083:     if args.freeze_real:
1084:         missing = [n for n, v in (("--trajectory", args.trajectory),
1085:                                   ("--labels", args.labels),
1086:                                   ("--fps", args.fps),
1087:                                   ("--frame-width", args.frame_width)) if v is None]
1092:         args.label, args.trajectory, args.labels,
1093:         license=args.license_ if args.license_ is not None else "",
1094:         minors_free=args.minors_free, owned=args.owned,
1095:         fps=args.fps, frame_width=args.frame_width,
1096:         fixtures_dir=fixtures_dir, slug=args.slug,
1097:         time_origin_reset=args.time_origin_reset, source_note=args.source_note)
===== backend/pipeline/detectors/wasb_infer.py =====
658:     ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
659:     ap.add_argument("--video", help="video file; frames are extracted internally (in WSL)")
660:     ap.add_argument("--frames_out_dir", help="where to extract frames when --video is used")
661:     ap.add_argument("--stream-video", action="store_true",
662:                     ap.add_argument("--weights", required=True)
663:                     ap.add_argument("--out", required=True)
664:                     ap.add_argument("--sport", default="badminton")
665:                     ap.add_argument("--limit", type=int, default=0, help="cap #frames (for quick tests)")
666:                     ap.add_argument("--batch-size", type=int, default=8)
667:                     ap.add_argument("--num-workers", type=int, default=4)
668:                     ap.add_argument("--log-every-batches", type=int, default=50, help="progress log
cadence")
671:     ap.add_argument("--cache-dir", default=None,
672:                     ap.add_argument("--flush-every", type=int, default=1000,
673:                     ap.add_argument("--fresh", action="store_true", help="ignore any existing cache and
start over")
676:     args = ap.parse_args()
677:     if args.stream_video:
678:         if not args.video:
679:             run_video_streaming(args.video, args.weights, args.out, args.sport, args.limit,
680:                                 batch_size=args.batch_size,
681:                                 log_every_batches=args.log_every_batches,
cache_dir=args.cache_dir,
682:                                 flush_every=args.flush_every, fresh=args.fresh)
683:     frames_dir = args.frames_dir
684:     if args.video:
685:         frames_dir = extract_frames(args.video, args.frames_out_dir or (args.video +
"_frames"))
686:     run(frames_dir, args.weights, args.out, args.sport, args.limit,
687:         batch_size=args.batch_size, num_workers=args.num_workers,
688:         log_every_batches=args.log_every_batches, cache_dir=args.cache_dir,
689:         flush_every=args.flush_every, fresh=args.fresh)
===== backend/eval/rally_seq_proto.py =====
142:         "n_pred": len(segs), "n_gt": len(v["gts"]),
143:     ap.add_argument("--manifest", required=True, help="JSON:
[{'name',trajectory,fps,frame_width,labels}, ...]")
144:     args = ap.parse_args()
145:     with open(args.manifest, encoding="utf-8") as f:

```

46. assistant (2026-06-18T01:35:21.530Z)

These all look clean. The `len(segs)` in `export\_wasb\_segments.py` line 152 operates on a local list (not a `process\_video` result), so it's fine. The `len(segs)` in `rally\_seq\_proto.py` line 142 also operates on local segment lists.

My primary finding stands. Let me do one more verification on the FastAPI side ? checking the `process\_video` endpoint order of validation vs file existence, and the `compile\_highlights` `segment\_ids` handling. Let me also double-check the `upload\_part` byte-accounting edge case once more, specifically the scenario where `received` overflow due to `part\_bytes` but `st["bytes"]` was already at limit.

Actually, let me re-examine `upload\_part` for a real bug: when the FIRST chunk already exceeds `part\_bytes`, it breaks, aborts, returns 413 ? correct. When chunks accumulate and `st["bytes"] + received > max\_bytes`, it breaks and aborts ? correct. The `received` counts the current chunk that triggered overflow even though it wasn't written, but since the upload is aborted entirely, the byte count is irrelevant. No bug there.

Let me verify the one concern about `validate\_input\_path` being called before `os.path.exists` in process ? but that's the correct order (validate format first, then existence). No bug.

Let me confirm there are no other callers of `process\_video` that mishandle the result outside main.py (which handles it correctly).

47. user (2026-06-18T01:35:24.286Z)

```
scripts/run_phase3_eval.py:145:         segs = segmenter.process_video(video_id,
video_path, max_frames=args.max_frames)
scripts/run_process_and_stitch.py:81:         segments = segmenter.process_video(video_id,
video_path)
tools/generate_highlights.py:167:         result = seg_cls(db, cast(Dict[str, Any],
cfg)).process_video(vid, proc_detect)
backend/main.py:192:         result = segmenter.process_video(video_id, req.video_path,
req.player_pool, req.max_frames)
backend/main.py:835:         result = segmenter.process_video(video_id, args.video_path,
max_frames=args.max_frames)
```

48. assistant (2026-06-18T01:35:27.904Z)

Let me check `run\_process\_and\_stitch.py` line 81 too ? it does `segments = segmenter.process\_video(...)` then likely uses `segments` directly. Let me check both that and confirm `generate\_highlights.py` handles it (it checks `isinstance(result, Failure)` at line 168, but then reads from DB at line 173, not from `result` ? so it's safe even for Success).

49. user (2026-06-18T01:35:28.960Z)

```
75         segmenter_cls = segmenter_registry.get(seg_name)
76         if not segmenter_cls:
77             print(f"[FAIL] Segmenter '{seg_name}' not found.")
78             return
79         print(f"Using segmenter: {seg_name}", flush=True)
80         segmenter = segmenter_cls(db, cfg)
81         segments = segmenter.process_video(video_id, video_path)
82         print(f"Successfully indexed {len(segments)} play segments.", flush=True)
83
84     if not segments:
85         print("No play segments detected. Cannot compile highlights.")
86         return
87
88
89     end_padding = cfg.stitching.end_padding
90     begin_padding = cfg.stitching.begin_padding
91     min_shot_count = cfg.stitching.min_shot_count
92
93     if cache_hit:
94         print("\n--- Stitching Customizations ---")
95         try:
96             bpad_input = input(f"Enter begin_padding in seconds (default
{begin_padding}): ").strip()
97             if bpad_input:
98                 begin_padding = float(bpad_input)
99
100             pad_input = input(f"Enter end_padding in seconds (default {end_padding}):
").strip()
101             if pad_input:
102                 end_padding = float(pad_input)
```

```

103
104         shot_input = input(f"Enter min_shot_count to filter by (default
{min_shot_count}): ").strip()
105         if shot_input:
106             min_shot_count = int(shot_input)
107     except ValueError:
108         print("[WARNING] Invalid input detected, falling back to defaults from
config.json.")
109
110     print(f"\nCompiling highlights (with begin_padding={begin_padding}s,
end_padding={end_padding}s, min_shot_count={min_shot_count})...", flush=True)
111
112     # Filter segments where shot_count >= min_shot_count
113     time_pairs = []
114     skipped_count = 0

```

50. assistant (2026-06-18T01:35:33.796Z)

This is a second instance of the same bug. `scripts/run\_process\_and\_stitch.py` line 82 does `len(segments)` directly on the `process\_video` return value, and line 84 does `if not segments:`. With the motion segmenter (which returns `Success`), `len(Success(...))` raises `TypeError` at line 82. And `if not segments:` on a `Success` object would always be falsy-evaluated as truthy (dataclass instances are truthy), so even if line 82 didn't crash, the empty-segments guard would be broken.

Let me check the top of this file to confirm the segmenter default and whether this is a `cache\_hit` only path or always runs.

51. user (2026-06-18T01:35:34.375Z)

```

1     import os
2     import json
3     import logging
4     from dotenv import load_dotenv
5
6     load_dotenv()
7
8     # Central root-logger setup ? all backend modules use logging.getLogger(__name__).
9     # Quiet third-party HTTP chatter by default; re-honoured against the config knob
10    # (logging.verbose_http) once the config is loaded in main().
11    from backend.utils.logging_setup import setup_logging
12    setup_logging()
13
14    from backend.infrastructure.database import Database
15    from backend.config import load_config
16    from backend.utils.hashing import compute_video_id
17    from backend.pipeline.segmenters import segmenter_registry
18    from backend.pipeline.stitcher.compiler import VideoCompiler
19    from backend.tools.export import format_rally_summary
20
21    logger = logging.getLogger(__name__)
22
23    def main():
24        video_path = r"C:\Users\avidu\Videos\Testing\KushagraYashAviFirstVideo.mp4"
25        output_dir = r"C:\Users\avidu\Videos\Testing"
26        output_filename = "Final_2_OutputOfKushagraYashAviFirstVideo.mp4"
27
28        print("Loading config...")
29        cfg = load_config() # typed AppConfig ? single source of defaults
30        # Re-apply now that the verbose_http knob is known (default keeps HTTP chatter
quiet).
31        setup_logging(verbose_http=cfg.logging.verbose_http)
32
33        print("Verifying files exist...")
34        if not os.path.exists(video_path):

```

```

35         print(f"Error: Video file not found at {video_path}")
36         return
37
38     os.makedirs(cfg.output.output_dir, exist_ok=True)
39     db_path = os.path.join(cfg.output.output_dir, cfg.output.db_name)
40     db = Database(db_path)
41
42     # Calculate MD5 of the video to uniquely identify it
43     import time
44     print("Calculating video MD5 (this might take a few seconds on large files)...",
45 flush=True)
46     t0 = time.time()
47     video_id = compute_video_id(video_path)
48     print(f"Calculated MD5: {video_id} (took {time.time()-t0:.1f}s)", flush=True)
49
50     segments = []
51     cached_video = db.get_video(video_id)
52     cache_hit = False
53
54     if cached_video and cached_video.get("status") == "processed":
55         segments = db.query_play_segments(video_id, {})
56         if len(segments) > 0:
57             print(f"\n[CACHE] Found cached indexed video in database (MD5: {video_id})
58 with {len(segments)} parsed play segments.", flush=True)
59             choice = input("Would you like to (1) Run stitching on the cached segments,
60 or (2) Reprocess the video with Gemini from scratch? [1/2]: ").strip()
61             if choice == '1':
62                 print("[CACHE] Using cached play segments. Skipping segmenter API
63 call.", flush=True)
64                 cache_hit = True
65             else:
66                 print("[CACHE] Ignoring cache...", flush=True)
67             else:
68                 print("[CACHE] Video exists in DB but contains 0 play segments. Treating as
69 cache miss.", flush=True)
70
71     if not cache_hit:
72         print("[CACHE] Initializing clean indexing...", flush=True)
73         db.clear_video_data(video_id)
74
75     print("Bypassing OpenCV validations for performance on massive files...",
76 flush=True)
77     db.add_video(video_id, video_path, "processed", [])
78
79     print("Running segmenter & indexing...", flush=True)
80     seg_name = cfg.indexing.default_segmenter
81     segmenter_cls = segmenter_registry.get(seg_name)

```

52. assistant (2026-06-18T01:35:38.017Z)

Now I need to determine the default segmenter. Line 74 uses `cfg.indexing.default\_segmenter`. Let me check what the config default is ? if it's `gemini` (returns a bare list), the bug wouldn't trigger for the default; but a user can change config to `motion`. Let me check the config default.

53. user (2026-06-18T01:35:39.788Z)

```

backend/config/models.py:118:     default_segmenter: str = "yolo_hybrid"
backend/config/models.py:192:     @field_validator("default_segmenter")
config.json:26:         "default_segmenter": "gemini",

```

54. assistant (2026-06-18T01:35:51.077Z)

So the default segmenter via config is `gemini` (config.json) / `yolo\_hybrid` (code default) ?

both return bare lists, so the bug in `run_process_and_stitch.py` is latent unless the user sets `default_segmenter` to `motion`. That makes this instance lower-confidence/lower-severity (requires config change to `motion`).

But for `run_phase3_eval.py`, the `--segmenter` default is explicitly `motion` (line 53), AND it's the documented "Free CPU baseline" example. So that instance is directly triggerable via the documented command. Higher confidence.

Let me reconsider severity. Both are real defects in the same class (mishandling the `Result` contract). Let me verify once more that `motion` is the one that returns `Success` and that the trajectory/yolo/gemini return bare lists (confirmed earlier from the grep: `gemini/yolo/trajectory` return `[]`/`segments_data` bare; only `motion` returns `Success`/`Failure`).

Now, one more consideration for `run_phase3_eval.py`: is there a chance `process_video` for `motion` is never reached because of `--score-only` or "already in DB"? The documented example on line 11-12 is a fresh run (`--only shuttle1`), which would process. So it's reachable.

I have two solid findings of the same bug class. Let me also reconsider whether `run_process_and_stitch.py` line 84 `if not segments:` is itself a bug independent of segmenter: for `gemini/yolo/trajectory` it returns a bare list, so `if not segments:` works correctly. For `motion` it returns `Success`, which is always truthy ? so the "No play segments detected" guard never triggers, AND `len()` crashes first anyway. Since the script's default is config-driven (gemini), I'll report it as medium/lower confidence.

Let me verify there isn't a `__len__`/`__bool__` somewhere on a base class. The `Success` dataclass has no such methods. Confirmed.

I have my findings. Let me record them.

55. user (2026-06-18T01:36:03.914Z)

Structured output provided successfully

56. assistant (2026-06-18T01:36:16.825Z)

I cross-cut by bug class (Result-contract / type-coercion mishandling at CLI entrypoints) across all CLI/argparse + FastAPI files. The FastAPI handlers in `backend/main.py` (upload byte-accounting, path validation, pagination/filter coercion, 404/400 paths, job store) were sound. I found two real instances of the same defect class ? CLI drivers calling `process_video` and then treating its `Result` return value (`Success`/`Failure` dataclass) as a plain list:

1. `**scripts/run_phase3_eval.py:145-146**` (medium, conf 0.85) ? `len(segs)` on the un-unwrapped result. The `--segmenter` default is `motion`, which returns `Success(value=...)` (`motion.py:224`), so the documented "Free CPU baseline" command crashes with `TypeError: object of type 'Success' has no len()`. `main.py` unwraps via `.value/instance(_, Failure)`; this driver does not. `gemini/yolo_hybrid/trajectory_hybrid` return bare lists and mask it, but `motion` (the default here) does not.

2. `**scripts/run_process_and_stitch.py:81-84**` (low, conf 0.70) ? same un-unwrapped result: `len(segments)` crashes and `if not segments:` is always truthy for a `Success`/`Failure` instance. Latent for the shipped default segmenter (gemini ? bare list) but a real defect if `default_segmenter` is set to `motion`.

Both fixes are the same: unwrap the `Result` (check `Failure`, then take `.value`) before measuring or truth-testing, mirroring `backend/main.py`.

Files of interest (all absolute):

- `C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/hunt-base/scripts/run_phase3_eval.py`
- `C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/hunt-base/scripts/run_process_and_stitch.py`
- `C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/hunt-base/backend/results.py` (the `Success`/`Failure` types)
- `C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/hunt-`

base/backend/pipeline/segmenters/motion.py` (returns `Success`/`Failure`)

I did not manufacture findings for the FastAPI request handlers ? they validate input, return correct 400/404/409/413 codes, and the upload/job paths are correct.